

3. 线性神经网络

3.1 线性回归

参考：《动手学深度学习》[3.1 线性回归](#)

回归 (regression) 是能为一个或多个自变量与因变量之间关系建模的一类方法。在自然科学和社会科学领域，回归经常用来表示输入和输出之间的关系。

在机器学习领域中，大多数任务都与**预测**有关。当我们想预测一个**数值**时，就会涉及回归问题。常见的例子包括：

- 预测价格（房屋、股票等）
- 预测住院时间
- 预测体育赛事胜负概率（对数回归）

本文以房价预测为例：我们希望根据房屋的**面积**（平方英尺）和**房龄**（年）来估算房屋的**价格**（美元）。为了实际拟合这个模型，我们需要收集真实数据集——一栋房子的实际售价、面积和房龄。于是机器学习的术语如下：

术语	英文	含义
训练数据集 / 训练集	training data set / training set	用来拟合模型参数的真实数据
样本 / 数据点	sample / data point	数据集中的每一行数据
标签 / 目标	label / target	要预测的内容（这里是价格）
特征 / 协变量	feature / covariate	预测所依据的自变量（这里是面积和房龄）

一个线性回归模型的完整要素包括：**模型**（线性模型）、**损失函数**、**优化算法**（随机梯度下降）。下面逐一展开。

1. 线性模型

线性假设是指：目标（价格 price）可以表示为特征（面积 area 和房龄 age）的**加权和**：

$$\text{price} = w_{\text{area}} \cdot \text{area} + w_{\text{age}} \cdot \text{age} + b$$

- w_{area} 和 w_{age} 称为**权重** (weight) 。权重越大，说明对应特征对预测结果的影响越大。
- b 称为**偏置** (bias) 、偏移量 (offset) 或**截距** (intercept) 。当所有特征都取 0 时，偏置就是预测值，表示价格的“基础水平”。

一般形式：给定数据集 $\{\mathbf{x}^{(1)}, \dots, \mathbf{x}^{(n)}\}$ (n 个样本，每个样本有 d 个特征)，线性模型的预测为：

$$\hat{y} = w_1x_1 + \cdots + w_dx_d + b$$

向量形式（点积）：

$$\hat{y} = \mathbf{w}^\top \mathbf{x} + b, \quad \mathbf{w}^\top \mathbf{x} = \sum_{i=1}^d w_i x_i$$

其中 $\mathbf{x} \in \mathbb{R}^d$ 是特征向量， $\mathbf{w} \in \mathbb{R}^d$ 是权重向量。

矩阵形式：对整个数据集，把 n 个样本按行堆叠成设计矩阵 $\mathbf{X} \in \mathbb{R}^{n \times d}$ ，预测向量 $\hat{\mathbf{y}} \in \mathbb{R}^n$ 为：

$$\hat{\mathbf{y}} = \mathbf{X}\mathbf{w} + b$$

- 通过矩阵-向量乘法，一行样本对应一个预测值，一次矩阵运算即可完成全部 n 个样本的预测（矢量化，见后文）。
- 偏置 b 会借助**广播机制**加到每一个样本的预测上。

给定特征 $\mathbf{X}$ ，目标是寻找权重向量 $\mathbf{w}$ 和偏置 b ，使得 $\hat{\mathbf{y}}$ 尽可能接近真实标签 $\mathbf{y}$ 。

严格来说， $\mathbf{X}\mathbf{w} + b$ 这种加入标量参数的做法属于**仿射变换**（affine transformation）——带有偏置项的线性变换。深度网络之所以“深”，就是因为在仿射变换之后叠加了多层非线性变换。

2. 损失函数（Loss Function）

在开始思考如何**拟合**（fit）模型之前，我们需要决定如何衡量模型的拟合质量。

损失函数（loss function）能够量化目标的**实际值与预测值之间的差距**。通常我们选择非负数作为损失，且数值越小表示损失越小，完美预测时损失为 0。

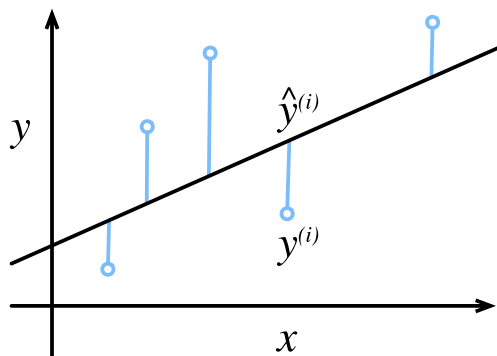
回归问题最常用的损失函数是**平方误差函数**（squared error function，也称 **L2 损失**）。当样本 i 的预测值为 $\hat{y}^{(i)}$ 、真实标签为 $y^{(i)}$ 时，平方误差定义为：

$$l^{(i)}(\mathbf{w}, b) = \frac{1}{2} \left(\hat{y}^{(i)} - y^{(i)} \right)^2$$

为什么有系数 $\frac{1}{2}$? 它对优化结果没有任何影响（只是缩放），但求导后刚好把平方的 2 消掉，使梯度形式更简洁：

$$\frac{\partial}{\partial \hat{y}^{(i)}} \frac{1}{2} \left(\hat{y}^{(i)} - y^{(i)} \right)^2 = \hat{y}^{(i)} - y^{(i)}$$

下图中，蓝色直线就是用线性模型拟合数据的结果，样本点到直线的**竖直距离**就是各样本的平方误差（残差）——直线越贴近数据点，损失越小。



训练损失（经验风险）： 由于平方误差对较大的差异惩罚更重，训练时我们希望寻找一组参数 $(\mathbf{w}^*, b^*)$ ，最小化所有训练样本上的平均损失：

$$L(\mathbf{w}, b) = \frac{1}{n} \sum_{i=1}^n l^{(i)}(\mathbf{w}, b) = \frac{1}{2n} \sum_{i=1}^n \left(\mathbf{w}^\top \mathbf{x}^{(i)} + b - y^{(i)} \right)^2$$

优化目标：

$$\mathbf{w}^*, b^* = \underset{\mathbf{w}, b}{\operatorname{argmin}} L(\mathbf{w}, b)$$

MSE 与 L1 损失的对比：

	均方误差 MSE (L2)	绝对误差 MAE (L1)
定义	$\frac{1}{2}(\hat{y} - y)^2$	$\ \hat{y} - y\ $
梯度	$\hat{y} - y$ (误差越大梯度越大)	± 1 (恒定)
对异常值	敏感 (平方放大 outlier 的影响)	稳健
最优点附近	梯度逐渐减小，收敛平稳	梯度不减小，最小值处不可导

回归中 MSE 是默认选择，而训练数据中的异常值（如测量错误）会对 MSE 影响很大——此时 L1 损失更稳健。

2.1 解析解 (Analytical Solution)

线性回归的损失 $L(\mathbf{w}, b)$ 是**凸函数**，属于"简单的优化问题"，最优解可以用公式直接表达出来，这类解叫**解析解**。

推导： 为方便矩阵运算，把偏置 b 吸收进权重向量——给每个样本增加一个恒为 1 的特征列，得到增广设计矩阵 $\mathbf{X} \in \mathbb{R}^{n \times (d+1)}$ ，增广权重向量 $\mathbf{w}$ (含 b)。此时损失可写为矩阵形式：

$$L(\mathbf{w}) = \frac{1}{2n} \|\mathbf{X}\mathbf{w} - \mathbf{y}\|^2$$

第一步：求梯度 (利用矩阵微积分 $\nabla_{\mathbf{w}} \|\mathbf{X}\mathbf{w} - \mathbf{y}\|^2 = 2\mathbf{X}^\top (\mathbf{X}\mathbf{w} - \mathbf{y})$)：

$$\frac{\partial L}{\partial \mathbf{w}} = \frac{1}{n} \mathbf{X}^\top (\mathbf{X} \mathbf{w} - \mathbf{y})$$

第二步：令梯度为零，得到正规方程（normal equations）：

$$\mathbf{X}^\top \mathbf{X} \mathbf{w} = \mathbf{X}^\top \mathbf{y}$$

第三步：当 $\mathbf{X}^\top \mathbf{X}$ 可逆时，最优解为：

$$\mathbf{w}^* = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y}$$

局限性：

1. 并不是所有问题都有解析解。解析解只对线性回归这类简单问题有效；深度神经网络根本没有解析解。
2. 即使存在解析解，当特征维度 d 很大时，矩阵求逆的代价约为 $\mathcal{O}(d^3)$ ，可能比迭代优化更昂贵。

因此实践中更常用的方法是**数值优化的迭代方法**——梯度下降。

3. 随机梯度下降（Stochastic Gradient Descent, SGD）

3.1 梯度下降的直觉

损失 $L(\mathbf{w}, b)$ 关于参数的梯度 $\nabla_{(\mathbf{w}, b)} L$ 指向损失**上升最快**的方向，那么它的**反方向**就是下降最快的方向。于是从随机初始化出发，沿负梯度方向一小步一小步地走：

$$\mathbf{w} \leftarrow \mathbf{w} - \eta \nabla_{\mathbf{w}} L(\mathbf{w}, b), \quad b \leftarrow b - \eta \frac{\partial L}{\partial b}$$

其中 $\eta > 0$ 是**学习率**（learning rate），控制每一步走多远。

问题：每次更新都要对**全部 n 个训练样本**求和来计算梯度。当数据集非常大时，单次更新的代价就非常高，甚至整个数据集都装不进内存。

3.2 随机梯度下降（SGD）

SGD 的核心思想：每次只用一个（或一小批）随机抽取的样本来估计梯度。

最朴素的（单样本）随机梯度下降：

$$\mathbf{w} \leftarrow \mathbf{w} - \eta \nabla_{\mathbf{w}} l^{(i)}(\mathbf{w}, b), \quad i \sim \text{Uniform}\{1, \dots, n\}$$

为什么这是合理的？——无偏估计。随机抽取的样本 i 的梯度是全体样本平均梯度的**无偏估计**（unbiased estimate）：

$$\mathbb{E}_i \left[\nabla_{\mathbf{w}} l^{(i)}(\mathbf{w}, b) \right] = \frac{1}{n} \sum_{i=1}^n \nabla_{\mathbf{w}} l^{(i)}(\mathbf{w}, b) = \nabla_{\mathbf{w}} L(\mathbf{w}, b)$$

即：虽然每次只看一个样本，但**平均而言**它指的方向和完整梯度一样，只是方差较大、路径"抖动"。

3.3 小批量随机梯度下降 (minibatch SGD) —— 实践标准

单样本更新太"抖"，且逐样本计算难以充分利用 GPU 的并行能力。实践中的标准做法是折中：每次迭代抽取**一小批量** (minibatch) 样本 $\mathcal{B}$ ，用批量内样本的平均梯度更新参数。

算法流程：

1. 初始化模型参数 $(\mathbf{w}_0, b_0)$ (通常随机初始化) ；
2. 重复以下步骤直到收敛：
 - 从训练集中随机抽取一个小批量 $\mathcal{B}_t = \{(\mathbf{x}_1, y_1), \dots, (\mathbf{x}_B, y_B)\}$ ，其中 $B = |\mathcal{B}|$ 称为**批量大小** (batch size) ；
 - 计算批量内损失的平均梯度；
 - 沿负梯度方向更新参数。

参数更新公式 (∂ 表示对 $(\mathbf{w}, b)$ 求梯度)：

$$(\mathbf{w}, b) \leftarrow (\mathbf{w}, b) - \frac{\eta}{|\mathcal{B}|} \sum_{i \in \mathcal{B}_t} \partial_{(\mathbf{w}, b)} l^{(i)}(\mathbf{w}, b)$$

无偏性：小批量梯度同样是无偏估计，且批量越大方差越小 (方差以 $1/B$ 的速率减小)：

$$\mathbb{E}_{\mathcal{B}} \left[\frac{1}{|\mathcal{B}|} \sum_{i \in \mathcal{B}} \nabla l^{(i)}(\mathbf{w}, b) \right] = \nabla L(\mathbf{w}, b)$$

关于采样方式：理论上"有放回"抽样与"无放回"抽样都可行；有放回抽样时上式的无偏性严格成立，便于理论分析。实践中通常先随机打乱全部样本、再按顺序切分成小批量 (相当于无放回)，每个 epoch 重新打乱一次。

3.4 两个关键超参数

超参数	含义	太小	太大
学习率 η	每步沿负梯度走多远	收敛缓慢，浪费算力	跨过最优点，震荡甚至发散
批量大小 B	每次更新使用的样本数	硬件并行利用率低、梯度噪声大	显存占用高、单次更新昂贵、可能被个别糟糕方向带偏

实践要点：

- 批量大小的选择常受**GPU 显存**限制；
- 批量内计算在 GPU 上**高度并行**：把批量从 B 扩大到 $2B$ ，几乎不会增加单次更新的耗时——因此在**算力允许时选大一点的批量**，并相应**放大学习率** (批量越大、梯度方差越小，可以容忍更大的步长) ；

- **收敛性质**：对于凸损失函数，若梯度范数有界且学习率满足一定条件（如 $\eta_t \propto 1/\sqrt{t}$ ），SGD 在期望意义下以 $\mathcal{O}(1/\sqrt{t})$ 的速率收敛到全局最优；对深度网络这样的非凸问题，SGD 同样是最主流的优化器；
- 实践中学习率常按"epoch"分段衰减，而不是严格按 $1/\sqrt{t}$ 衰减。

3.5 用模型进行预测（推断）

训练完成后，可以用学到的参数 $\hat{\mathbf{w}}, \hat{b}$ 对**新样本**做预测（也称**推断**，inference）。例如给定一套新房的面积 $\mathbf{x}$ ，预测价格为：

$$\hat{y} = \hat{\mathbf{w}}^\top \mathbf{x} + \hat{b}$$

由于这里是我们**手工挑选**的特征（面积、房龄），这个练习其实属于标准统计学的范畴，还谈不上真正的"学习"。后面章节会看到，深度学习可以自动学习特征表示。

3.6 动手验证：小批量 SGD 从零实现

用带噪声的线性模型生成合成数据 $y = 2x_1 - 3.4x_2 + 4.2 + \epsilon$ ，再用上一节的小批量 SGD 把参数"学"回来。

```
In [63]: import random
import torch

def synthetic_data(w, b, num_examples):
    """生成  $y = Xw + b + \text{噪声}$ 。"""
    X = torch.normal(0, 1, (num_examples, len(w)))
    y = torch.matmul(X, w) + b
    y += torch.normal(0, 0.01, y.shape) # 加上均值为0、标准差0.01的高斯噪声
    return X, y.reshape((-1, 1))

true_w = torch.tensor([2.0, -3.4])
true_b = 4.2
features, labels = synthetic_data(true_w, true_b, 1000)

print('X 的形状:', features.shape, ' y 的形状:', labels.shape)
print('第一个样本:', features[0], ' 对应标签:', labels[0])
```

X 的形状: torch.Size([1000, 2]) y 的形状: torch.Size([1000, 1])
 第一个样本: tensor([2.0709, -0.8109]) 对应标签: tensor([11.0884])

```
In [64]: def data_iter(batch_size, X, y):
    """小批量数据迭代器：每个 epoch 随机打乱样本，按批量大小切分。"""
    num_examples = len(X)
    indices = list(range(num_examples))
    random.shuffle(indices) # 随机打乱，保证批量内样本是随机抽取的
    for i in range(0, num_examples, batch_size):
        batch_indices = torch.tensor(indices[i : min(i + batch_size, num_examples)])
        print("batch_indices:", batch_indices)
        yield X[batch_indices], y[batch_indices]

batch_size = 10
```

```

for X_b, y_b in data_iter(batch_size, features, labels):
    print("第一个批量的特征:\n", X_b[:3])
    print("第一个批量的标签:\n", y_b[:3])
    break

```

batch_indices: tensor([891, 957, 11, 529, 691, 426, 615, 642, 237, 256])

第一个批量的特征:

```

tensor([[ 1.9451, -1.1039],
        [-0.6218, -0.6421],
        [-1.4977,  0.0358]])

```

第一个批量的标签:

```

tensor([[11.8325],
        [ 5.1438],
        [ 1.0669]])

```

```

In [71]: # 初始化参数（从正态分布中随机取 w, b 置 0），并开启梯度追踪
w = torch.normal(0, 0.01, size=(2, 1), requires_grad=True)
b = torch.zeros(1, requires_grad=True)

def linreg(X, w, b):
    """线性回归模型: y = Xw + b。"""
    return torch.matmul(X, w) + b

def squared_loss(y_hat, y):
    """平方损失（逐样本，未求均值）。"""
    return (y_hat - y.reshape(y_hat.shape)) ** 2 / 2

lr = 0.03          # 学习率 η
num_epochs = 3     # 完整遍历数据集的轮数
net = linreg
loss = squared_loss

for epoch in range(num_epochs):
    for X_b, y_b in data_iter(batch_size, features, labels):
        l = loss(net(X_b, w, b), y_b) # 小批量损失，形状 (batch_size, 1)
        print(f'l:{l}')
        l.sum().backward()             # 对 (w, b) 反向传播求梯度
        with torch.no_grad():          # 参数更新不需要参与梯度计算
            # 对应公式: (w, b) <- (w, b) - η/|B| * Σ ∂L
            w -= lr * w.grad / batch_size
            b -= lr * b.grad / batch_size
            w.grad.zero_()              # 梯度清零，为下一次迭代做准备
            b.grad.zero_()
        train_l = loss(net(features, w, b), labels)
        print(f'epoch {epoch + 1}, 平均损失 {train_l.mean():f}')

print(f'w 的估计误差: {true_w - w.reshape(true_w.shape):f}')
print(f'b 的估计误差: {true_b - b:f}')

```

```

batch_indices: tensor([234, 68, 553, 850, 115, 845, 132, 852, 743, 602])
l:tensor([[ 3.0471],
          [11.9249],
          [ 7.5963],
          [ 3.7813],
          [ 5.4034],
          [30.5951],
          [12.8185],
          [ 0.7349],
          [15.9040],
          [76.5101]], grad_fn=<DivBackward0>)
batch_indices: tensor([939, 479, 290, 884, 800, 220, 821, 842, 551, 664])
l:tensor([[ 0.0741],
          [ 4.1578],
          [32.2800],
          [25.9676],
          [ 9.3928],
          [ 2.5731],
          [ 2.5689],
          [25.3312],
          [42.3590],
          [16.3488]], grad_fn=<DivBackward0>)
batch_indices: tensor([706, 126, 459, 381, 205, 690, 428, 545, 251, 552])
l:tensor([[ 0.1084],
          [ 5.0797],
          [23.1065],
          [ 0.0269],
          [ 0.1407],
          [ 1.2780],
          [10.4161],
          [ 0.3420],
          [ 3.9613],
          [ 2.0154]], grad_fn=<DivBackward0>)
batch_indices: tensor([324, 843, 461, 890, 368, 868, 491, 73, 11, 288])
l:tensor([[5.9234e+00],
          [1.1820e+00],
          [2.0960e-02],
          [7.4474e-01],
          [2.9864e+00],
          [2.9165e+01],
          [5.6945e-01],
          [5.7380e+00],
          [4.8818e-01],
          [1.4585e+01]], grad_fn=<DivBackward0>)
batch_indices: tensor([294, 214, 188, 460, 753, 750, 347, 990, 336, 295])
l:tensor([[14.6610],
          [ 4.8218],
          [12.5701],
          [13.8337],
          [ 4.3710],
          [15.1681],
          [11.4288],
          [ 8.0705],
          [19.9814],
          [ 5.8690]], grad_fn=<DivBackward0>)
batch_indices: tensor([136, 726, 751, 50, 225, 893, 772, 841, 748, 985])

```



```

1:tensor([[25.8566],
          [29.4761],
          [ 3.5750],
          [18.3953],
          [68.5420],
          [54.5079],
          [ 0.8361],
          [76.2744],
          [ 3.8514],
          [ 7.0592]], grad_fn=<DivBackward0>)
batch_indices: tensor([292, 739, 891, 362, 241, 997, 277, 165,  6, 769])
1:tensor([[1.8010e+01],
          [2.5206e+01],
          [4.9314e+01],
          [1.1285e+01],
          [3.7561e+00],
          [2.3687e+01],
          [1.8907e-02],
          [1.4300e+01],
          [6.6494e+00],
          [8.6093e+00]], grad_fn=<DivBackward0>)
batch_indices: tensor([631, 149, 482, 301, 790, 653, 218, 280, 798, 303])
1:tensor([[33.1943],
          [ 1.4118],
          [ 9.8014],
          [ 6.4526],
          [ 3.7007],
          [10.5721],
          [ 4.6525],
          [ 7.5957],
          [ 4.6567],
          [ 0.2823]], grad_fn=<DivBackward0>)
batch_indices: tensor([217, 111, 498, 378, 342, 612, 991, 949, 578, 420])
1:tensor([[ 3.0619],
          [17.8313],
          [ 4.8580],
          [ 4.1567],
          [ 2.0826],
          [52.4153],
          [ 6.2905],
          [ 4.9280],
          [ 2.8480],
          [ 0.8636]], grad_fn=<DivBackward0>)
batch_indices: tensor([257, 574, 352, 185, 181, 777, 556, 846, 801, 809])
1:tensor([[2.9633e+01],
          [8.5812e-02],
          [3.4821e+00],
          [2.2708e-02],
          [5.1265e+00],
          [9.1727e-01],
          [8.6602e-01],
          [1.2389e+01],
          [3.0724e+01],
          [1.0488e+01]], grad_fn=<DivBackward0>)
batch_indices: tensor([870, 159, 960, 810, 366, 875, 730, 649, 463, 252])
1:tensor([[5.0216e+00],

```

```

[2.3603e-01],
[3.5638e+00],
[2.4520e+00],
[2.9838e+00],
[1.4532e+00],
[3.0590e+01],
[8.1484e+00],
[1.0410e+01],
[1.0175e-02]], grad_fn=<DivBackward0>)
batch_indices: tensor([108, 797, 284, 727, 541, 448, 285, 864, 565, 585])
l:tensor([[5.1276e-03],
[9.9806e+00],
[1.3744e+00],
[2.2672e+00],
[5.7429e+00],
[7.1716e+00],
[7.0191e+00],
[2.9911e+01],
[1.3412e+00],
[4.0454e+00]], grad_fn=<DivBackward0>)
batch_indices: tensor([510, 932, 513, 487, 831, 999, 445, 31, 916, 135])
l:tensor([[12.9581],
[ 1.7592],
[11.0148],
[ 0.5076],
[ 9.2732],
[ 9.5817],
[16.4882],
[ 1.6214],
[15.5903],
[ 3.5389]], grad_fn=<DivBackward0>)
batch_indices: tensor([642, 509, 256, 198, 102, 91, 765, 947, 712, 486])
l:tensor([[8.4364e+00],
[1.9633e+01],
[9.6923e-01],
[6.8534e+00],
[1.0776e+01],
[5.8528e+00],
[1.4534e+00],
[2.7882e+01],
[1.1038e+01],
[2.5947e-02]], grad_fn=<DivBackward0>)
batch_indices: tensor([118, 95, 52, 961, 7, 851, 709, 194, 146, 767])
l:tensor([[25.5753],
[ 9.5893],
[ 3.3441],
[ 0.0690],
[18.7264],
[29.1770],
[ 6.0196],
[ 1.4536],
[ 0.4288],
[ 8.0427]], grad_fn=<DivBackward0>)
batch_indices: tensor([438, 784, 483, 82, 248, 150, 781, 855, 206, 737])
l:tensor([[ 8.1789],
[35.4115],

```

```

[ 9.8149],
[ 0.4784],
[15.4825],
[ 8.7838],
[ 0.1761],
[ 3.4951],
[ 2.7994],
[ 4.4218]], grad_fn=<DivBackward0>)
batch_indices: tensor([686, 854, 203, 527, 228, 865, 423, 191, 338, 358])
l:tensor([[ 2.3031],
[ 2.5025],
[ 8.6865],
[ 2.9675],
[ 1.5042],
[ 2.1580],
[ 4.6496],
[16.8563],
[18.2485],
[ 0.0245]], grad_fn=<DivBackward0>)
batch_indices: tensor([481, 522, 97, 792, 401, 927, 506, 502, 918, 819])
l:tensor([[ 5.4603],
[24.7966],
[10.7096],
[ 0.3908],
[12.3670],
[ 0.1603],
[ 2.2059],
[16.2825],
[ 0.8074],
[10.3943]], grad_fn=<DivBackward0>)
batch_indices: tensor([372, 713, 637, 538, 745, 979, 935, 657, 684, 478])
l:tensor([[2.0482e+00],
[5.0653e-01],
[9.9018e+00],
[1.9019e-02],
[5.5605e+00],
[3.9288e+00],
[3.4200e+01],
[1.1206e-01],
[4.6302e+00],
[1.2438e+00]], grad_fn=<DivBackward0>)
batch_indices: tensor([151, 260, 143, 311, 38, 168, 550, 834, 47, 791])
l:tensor([[1.2731e+00],
[7.6817e-02],
[9.3887e-03],
[1.1583e-04],
[1.3938e+00],
[2.4275e-02],
[8.0288e-04],
[1.1501e-01],
[6.0873e+00],
[1.0543e+01]], grad_fn=<DivBackward0>)
batch_indices: tensor([410, 641, 838, 77, 973, 862, 9, 524, 908, 705])
l:tensor([[3.7573],
[0.2787],
[0.4745],

```

```

        [7.0826],
        [4.8367],
        [6.6441],
        [7.1899],
        [9.6872],
        [1.0651],
        [7.7890]], grad_fn=<DivBackward0>)
batch_indices: tensor([122, 511, 571, 237, 321, 344, 477, 833, 40, 882])
l:tensor([[ 8.0077],
        [19.9462],
        [11.7970],
        [ 2.1783],
        [ 5.3275],
        [ 2.5778],
        [ 0.3722],
        [ 8.2571],
        [ 0.8057],
        [ 6.8597]], grad_fn=<DivBackward0>)
batch_indices: tensor([505, 282, 49, 51, 110, 60, 385, 55, 204, 224])
l:tensor([[7.0743e+00],
        [2.2163e+00],
        [2.4614e-03],
        [8.7284e+00],
        [1.0166e+01],
        [5.6534e+00],
        [3.3407e+00],
        [1.7613e+00],
        [2.0240e+00],
        [5.9118e+00]], grad_fn=<DivBackward0>)
batch_indices: tensor([937, 570, 901, 923, 161, 968, 618, 229, 69, 337])
l:tensor([[ 5.4104],
        [ 5.3375],
        [ 1.0301],
        [25.7627],
        [ 9.2148],
        [ 0.3098],
        [ 3.8734],
        [ 3.3282],
        [ 5.0758],
        [ 4.1039]], grad_fn=<DivBackward0>)
batch_indices: tensor([803, 302, 529, 216, 931, 386, 685, 785, 848, 702])
l:tensor([[1.9153e-03],
        [4.6889e-01],
        [1.7924e+00],
        [2.6124e-01],
        [2.0747e+00],
        [1.9973e+00],
        [4.1583e-01],
        [5.0356e-02],
        [1.1774e+01],
        [9.7927e+00]], grad_fn=<DivBackward0>)
batch_indices: tensor([788, 215, 109, 471, 993, 683, 429, 390, 547, 728])
l:tensor([[ 0.2375],
        [ 1.4751],
        [ 3.2088],
        [ 1.2647],

```

```

[ 3.9462],
[30.4311],
[ 5.0004],
[ 7.9461],
[ 2.0547],
[ 1.8678]], grad_fn=<DivBackward0>)
batch_indices: tensor([128, 190, 808, 786, 66, 861, 964, 661, 34, 22])
l:tensor([[2.3550e+00],
[2.6725e+00],
[1.1032e+01],
[5.0109e+00],
[7.6145e+00],
[6.7520e-02],
[8.3995e-01],
[1.0525e+00],
[4.6550e+00],
[2.6730e-03]], grad_fn=<DivBackward0>)
batch_indices: tensor([548, 349, 633, 869, 467, 663, 407, 735, 496, 978])
l:tensor([[7.6274e-02],
[2.1002e+00],
[4.3130e+00],
[3.0579e-01],
[4.6205e+00],
[1.0694e-03],
[6.2823e-02],
[3.0482e-01],
[8.4276e+00],
[2.0217e+00]], grad_fn=<DivBackward0>)
batch_indices: tensor([501, 910, 44, 811, 304, 897, 222, 928, 853, 886])
l:tensor([[2.4486],
[3.9153],
[6.2401],
[0.4061],
[0.1899],
[4.9200],
[0.6835],
[0.3678],
[1.1736],
[2.6162]], grad_fn=<DivBackward0>)
batch_indices: tensor([422, 621, 174, 598, 887, 586, 863, 627, 970, 436])
l:tensor([[8.1250e+00],
[1.6113e+00],
[1.6795e+00],
[1.3654e+00],
[1.2660e+00],
[1.1119e+00],
[7.9526e-01],
[3.0567e+00],
[7.0119e-01],
[3.2434e-03]], grad_fn=<DivBackward0>)
batch_indices: tensor([307, 717, 391, 123, 983, 889, 361, 30, 223, 531])
l:tensor([[0.6424],
[0.3731],
[1.0655],
[0.0110],
[0.0919],

```

```

[5.5961],
[0.7022],
[2.5447],
[1.6186],
[8.0422]], grad_fn=<DivBackward0>)
batch_indices: tensor([599, 738, 365, 806, 895, 298, 456, 775, 874, 147])
l:tensor([[1.1548],
[6.8366],
[0.8521],
[0.1352],
[0.8884],
[4.9200],
[0.8839],
[0.0359],
[0.9857],
[2.6567]], grad_fn=<DivBackward0>)
batch_indices: tensor([ 1, 515, 341, 573, 101, 988, 659, 966, 90, 213])
l:tensor([[1.2621e+00],
[7.7797e-01],
[5.8801e-01],
[4.5713e-01],
[3.8108e-03],
[3.9035e-01],
[1.5043e-01],
[3.9633e-01],
[7.3498e-01],
[4.9651e+00]], grad_fn=<DivBackward0>)
batch_indices: tensor([903, 694, 900, 733, 701, 152, 462, 992, 320, 212])
l:tensor([[12.1461],
[ 2.2825],
[ 0.3689],
[ 2.8353],
[ 1.2284],
[ 0.6827],
[ 3.2158],
[ 1.5985],
[ 0.5569],
[ 0.2382]], grad_fn=<DivBackward0>)
batch_indices: tensor([ 14, 517, 113, 329, 177, 455, 193, 648, 208, 994])
l:tensor([[4.2325e-04],
[4.2987e+00],
[5.8428e-01],
[2.1646e-02],
[2.3087e+00],
[1.0489e+00],
[1.2924e-01],
[5.6973e+00],
[1.2313e+01],
[1.9671e-03]], grad_fn=<DivBackward0>)
batch_indices: tensor([711, 305, 404, 989, 560, 245, 13, 543, 308, 569])
l:tensor([[4.9772],
[0.2108],
[0.6081],
[2.2955],
[0.5604],
[4.7150],

```

```

[1.5504],
[1.4579],
[1.4549],
[0.3055]], grad_fn=<DivBackward0>)
batch_indices: tensor([433, 283, 723, 638, 458, 628, 528, 580, 61, 270])
l:tensor([[3.7516e-01],
[6.6288e-05],
[3.9410e-01],
[5.2350e-01],
[1.6546e+00],
[1.3878e+00],
[8.1614e-01],
[1.4555e-02],
[9.9926e-01],
[1.1667e+00]], grad_fn=<DivBackward0>)
batch_indices: tensor([823, 888, 470, 568, 432, 84, 698, 184, 732, 955])
l:tensor([[0.4789],
[1.1358],
[0.1530],
[0.9853],
[1.1586],
[7.5019],
[0.5011],
[1.4815],
[1.9013],
[0.5766]], grad_fn=<DivBackward0>)
batch_indices: tensor([389, 117, 96, 827, 59, 593, 33, 107, 12, 377])
l:tensor([[2.0867],
[1.1495],
[2.0921],
[0.1924],
[6.0940],
[1.3888],
[4.3037],
[0.6483],
[0.4966],
[1.6784]], grad_fn=<DivBackward0>)
batch_indices: tensor([582, 589, 691, 76, 822, 825, 169, 273, 409, 275])
l:tensor([[0.0640],
[0.8991],
[0.2418],
[0.2994],
[0.0803],
[1.4862],
[0.3326],
[0.2878],
[1.0878],
[2.1294]], grad_fn=<DivBackward0>)
batch_indices: tensor([364, 56, 442, 447, 180, 299, 116, 796, 756, 934])
l:tensor([[1.3431],
[0.0680],
[5.1129],
[0.5355],
[0.6744],
[0.2319],
[0.3088],

```

```

[1.9694],
[1.2429],
[1.1106]], grad_fn=<DivBackward0>)
batch_indices: tensor([594, 636, 263, 972, 235, 614, 714, 536, 885, 155])
l:tensor([[1.5589e-02],
[6.2906e+00],
[7.0876e-01],
[6.7934e-02],
[1.2340e+00],
[2.8743e+00],
[4.7070e-01],
[1.5869e+00],
[2.7188e-04],
[2.2866e+00]], grad_fn=<DivBackward0>)
batch_indices: tensor([170, 830, 114, 269, 46, 603, 878, 655, 154, 93])
l:tensor([[2.9600e-05],
[5.4645e+00],
[5.3687e+00],
[2.6248e-01],
[3.3543e-01],
[2.6019e+00],
[5.1964e+00],
[9.0013e-01],
[1.1982e+00],
[8.0830e-03]], grad_fn=<DivBackward0>)
batch_indices: tensor([933, 954, 356, 413, 253, 998, 629, 371, 88, 665])
l:tensor([[2.6065],
[0.1652],
[0.8841],
[0.3060],
[7.4783],
[0.5925],
[1.6645],
[0.3025],
[0.4151],
[1.0131]], grad_fn=<DivBackward0>)
batch_indices: tensor([611, 635, 121, 457, 317, 449, 671, 232, 982, 533])
l:tensor([[3.8110],
[2.9249],
[1.5287],
[0.0142],
[1.6767],
[0.5357],
[2.6493],
[0.2839],
[0.1943],
[1.5639]], grad_fn=<DivBackward0>)
batch_indices: tensor([587, 986, 86, 668, 466, 133, 57, 716, 19, 815])
l:tensor([[3.2336e-02],
[2.3236e+00],
[9.1264e-01],
[1.2103e+00],
[4.6387e-04],
[2.6135e-02],
[2.8113e+00],
[4.0980e-01],

```



```

[2.2982e-01],
[2.8038e-01]], grad_fn=<DivBackward0>)
batch_indices: tensor([879, 24, 525, 322, 106, 818, 196, 919, 74, 265])
l:tensor([[0.1316],
[0.0692],
[0.4691],
[5.2715],
[0.8140],
[0.0156],
[0.4579],
[1.4967],
[0.0347],
[2.2349]], grad_fn=<DivBackward0>)
batch_indices: tensor([129, 896, 674, 186, 221, 355, 10, 201, 789, 141])
l:tensor([[0.0374],
[0.1123],
[0.4892],
[3.4327],
[0.8647],
[1.8922],
[1.6994],
[0.3554],
[0.2983],
[0.6482]], grad_fn=<DivBackward0>)
batch_indices: tensor([490, 909, 600, 153, 773, 700, 451, 70, 281, 721])
l:tensor([[2.1842e-03],
[3.1423e+00],
[5.2667e-01],
[1.7305e-01],
[7.1066e-01],
[1.2386e+00],
[1.5156e+00],
[5.1425e-01],
[4.3456e-02],
[1.0302e-01]], grad_fn=<DivBackward0>)
batch_indices: tensor([339, 844, 718, 835, 431, 840, 549, 310, 588, 160])
l:tensor([[1.3319],
[1.6021],
[0.7069],
[0.0054],
[1.4754],
[2.8134],
[0.0216],
[0.1980],
[0.0857],
[0.1309]], grad_fn=<DivBackward0>)
batch_indices: tensor([246, 388, 23, 334, 374, 103, 239, 375, 492, 558])
l:tensor([[0.6701],
[0.1591],
[0.0531],
[0.1188],
[0.7995],
[0.4357],
[0.0729],
[0.4849],
[0.6039],

```

```

[0.0467]], grad_fn=<DivBackward0>)
batch_indices: tensor([640, 617, 326, 473, 626, 914, 667, 488, 493, 236])
l:tensor([[0.0416],
          [0.3686],
          [0.0726],
          [1.0305],
          [0.0321],
          [1.4215],
          [0.3171],
          [0.0096],
          [2.0108],
          [0.9460]], grad_fn=<DivBackward0>)
batch_indices: tensor([824, 163, 172, 262, 157, 925, 27, 542, 804, 622])
l:tensor([[0.0799],
          [1.0587],
          [0.9699],
          [0.7540],
          [0.2755],
          [1.2635],
          [1.0706],
          [0.1001],
          [1.5111],
          [0.0016]], grad_fn=<DivBackward0>)
batch_indices: tensor([171, 139, 166, 434, 98, 591, 771, 87, 335, 969])
l:tensor([[6.5943e-01],
          [2.1344e-04],
          [2.8365e+00],
          [1.5146e-01],
          [3.8113e-01],
          [1.0236e+00],
          [8.6574e-01],
          [1.8731e+00],
          [1.3068e-01],
          [4.9765e-01]], grad_fn=<DivBackward0>)
batch_indices: tensor([601, 376, 249, 446, 226, 367, 469, 512, 689, 620])
l:tensor([[1.2420e+00],
          [6.8036e-04],
          [1.0538e+00],
          [5.0631e-01],
          [1.3611e+00],
          [6.2803e-02],
          [1.9158e-01],
          [4.7421e-01],
          [2.2127e-02],
          [1.9410e-01]], grad_fn=<DivBackward0>)
batch_indices: tensor([583, 688, 894, 474, 577, 408, 131, 254, 562, 724])
l:tensor([[7.7257e-05],
          [2.1779e-02],
          [2.6883e+00],
          [1.5195e-01],
          [9.5219e-02],
          [7.3826e-04],
          [1.7499e+00],
          [6.9821e-01],
          [3.5824e-01],
          [1.5272e+00]], grad_fn=<DivBackward0>)

```

```
batch_indices: tensor([692, 315, 287, 485, 259, 357, 610, 678, 680, 518])
l:tensor([[3.1532e-01],
          [6.4250e-02],
          [8.0590e-06],
          [9.9049e-04],
          [3.7767e-02],
          [6.5002e-02],
          [3.6834e-04],
          [2.2505e-02],
          [2.1491e-02],
          [3.1225e-01]], grad_fn=<DivBackward0>)
batch_indices: tensor([348, 18, 679, 832, 981, 296, 521, 936, 20, 158])
l:tensor([[0.3254],
          [0.3828],
          [0.0611],
          [1.6071],
          [0.0051],
          [0.0073],
          [0.1005],
          [0.1396],
          [0.3804],
          [1.1042]], grad_fn=<DivBackward0>)
batch_indices: tensor([837, 774, 956, 207, 293, 351, 15, 297, 755, 3])
l:tensor([[0.6556],
          [0.0885],
          [0.4002],
          [0.9222],
          [1.1113],
          [0.3187],
          [0.0761],
          [0.3421],
          [0.1759],
          [0.3005]], grad_fn=<DivBackward0>)
batch_indices: tensor([584, 119, 836, 760, 695, 258, 904, 872, 849, 75])
l:tensor([[0.7718],
          [0.5294],
          [0.0344],
          [0.6378],
          [0.4817],
          [0.0011],
          [0.1090],
          [0.1994],
          [0.0009],
          [0.2017]], grad_fn=<DivBackward0>)
batch_indices: tensor([566, 176, 742, 920, 567, 660, 676, 929, 327, 343])
l:tensor([[0.5039],
          [0.9859],
          [0.0060],
          [0.5491],
          [0.1642],
          [0.3575],
          [0.1036],
          [0.7022],
          [1.1742],
          [0.1492]], grad_fn=<DivBackward0>)
batch_indices: tensor([440, 650, 162, 564, 148, 392, 707, 975, 25, 951])
```

```

l:tensor([[2.7253e+00],
          [1.9025e-02],
          [1.0857e-01],
          [3.3616e-05],
          [1.8641e-02],
          [7.1527e-02],
          [7.7372e-01],
          [1.3450e-01],
          [3.1838e-01],
          [1.6797e-01]], grad_fn=<DivBackward0>)
batch_indices: tensor([255, 606, 394, 100, 247, 274, 561, 350, 399, 189])
l:tensor([[0.0936],
          [0.2844],
          [0.0702],
          [0.0109],
          [0.2849],
          [0.1878],
          [1.1255],
          [0.1911],
          [0.1255],
          [0.1695]], grad_fn=<DivBackward0>)
batch_indices: tensor([369, 125, 83, 675, 417, 414, 915, 179, 652, 360])
l:tensor([[0.3702],
          [0.2445],
          [0.2279],
          [0.9828],
          [0.1763],
          [0.0238],
          [0.0895],
          [0.0667],
          [0.0128],
          [0.1670]], grad_fn=<DivBackward0>)
batch_indices: tensor([546, 316, 318, 778, 911, 134, 693, 670, 418, 278])
l:tensor([[7.9729e-01],
          [4.2016e-01],
          [2.5714e-02],
          [1.0558e+00],
          [3.7294e-01],
          [3.0568e-01],
          [4.1508e-05],
          [9.4510e-01],
          [3.8343e-01],
          [5.8253e-01]], grad_fn=<DivBackward0>)
batch_indices: tensor([480, 48, 411, 963, 72, 625, 632, 731, 145, 331])
l:tensor([[0.0545],
          [0.1486],
          [0.0041],
          [0.1177],
          [0.0353],
          [0.0012],
          [0.3621],
          [0.0944],
          [0.3311],
          [0.4426]], grad_fn=<DivBackward0>)
batch_indices: tensor([85, 156, 8, 396, 532, 647, 758, 977, 499, 427])
l:tensor([[0.4390],

```

```

[0.5155],
[0.3019],
[0.3103],
[0.1207],
[0.0647],
[0.2408],
[0.1344],
[0.3890],
[0.0716]], grad_fn=<DivBackward0>)
batch_indices: tensor([971, 572, 859, 656, 227, 178, 202, 645, 430, 906])
l:tensor([[0.2080],
[0.4704],
[0.2525],
[0.1145],
[0.1364],
[0.0166],
[0.3932],
[0.0029],
[0.4206],
[0.1181]], grad_fn=<DivBackward0>)
batch_indices: tensor([382, 757, 199, 805, 89, 415, 644, 658, 922, 639])
l:tensor([[0.0037],
[0.0227],
[0.6117],
[0.1524],
[1.0071],
[0.2365],
[0.0684],
[0.0304],
[0.2020],
[0.0120]], grad_fn=<DivBackward0>)
batch_indices: tensor([395, 332, 183, 416, 35, 397, 250, 130, 858, 624])
l:tensor([[5.2055e-01],
[1.4181e-01],
[4.2094e-04],
[7.1689e-02],
[3.1530e-04],
[1.2783e-01],
[1.1522e+00],
[1.1375e+00],
[3.5682e-03],
[6.7082e-02]], grad_fn=<DivBackward0>)
batch_indices: tensor([441, 877, 402, 902, 495, 94, 314, 729, 79, 535])
l:tensor([[0.1381],
[0.0181],
[0.1130],
[0.0100],
[0.4843],
[0.6929],
[0.1408],
[0.0982],
[0.6077],
[0.0140]], grad_fn=<DivBackward0>)
batch_indices: tensor([211, 539, 36, 465, 127, 261, 323, 426, 747, 167])
l:tensor([[0.1815],
[0.1246],

```

```

        [0.0822],
        [0.1693],
        [0.0049],
        [0.0115],
        [0.2682],
        [0.0006],
        [0.0274],
        [0.1035]], grad_fn=<DivBackward0>)
batch_indices: tensor([359, 734, 476, 266, 860, 759, 871, 319, 704, 398])
l:tensor([[0.0588],
        [0.4062],
        [0.1745],
        [0.3744],
        [0.0094],
        [0.2100],
        [0.2839],
        [0.1321],
        [0.0122],
        [0.0017]], grad_fn=<DivBackward0>)
batch_indices: tensor([ 78, 780, 609,  39, 802, 472, 662, 881, 557, 926])
l:tensor([[0.0816],
        [0.1284],
        [0.1445],
        [0.2786],
        [0.0895],
        [0.0048],
        [0.3516],
        [0.0018],
        [0.0182],
        [0.0309]], grad_fn=<DivBackward0>)
batch_indices: tensor([953, 124, 957, 744, 238,  37, 782, 787,  99, 497])
l:tensor([[0.0280],
        [0.0910],
        [0.1134],
        [0.2698],
        [0.1871],
        [0.2263],
        [0.5410],
        [0.0034],
        [0.0006],
        [0.5756]], grad_fn=<DivBackward0>)
batch_indices: tensor([ 62, 958, 795, 333, 597, 905, 267, 995,  32, 592])
l:tensor([[1.3459e-01],
        [6.3390e-02],
        [1.3461e-02],
        [1.8396e-03],
        [7.6225e-02],
        [3.8385e-05],
        [1.3902e-03],
        [1.3199e-01],
        [4.2021e-02],
        [6.7234e-02]], grad_fn=<DivBackward0>)
batch_indices: tensor([579, 736, 619,  17, 575, 175,  67, 503, 703, 813])
l:tensor([[3.4015e-02],
        [3.2107e-02],
        [5.1965e-01],

```

```

[2.5876e-02],
[1.2955e-01],
[1.1332e-01],
[1.0203e-01],
[8.0117e-05],
[5.8657e-02],
[1.4120e-01]], grad_fn=<DivBackward0>)
batch_indices: tensor([444, 268, 210, 219, 880, 425, 21, 761, 666, 520])
l:tensor([[0.1120],
[0.0876],
[0.0020],
[0.3285],
[0.2937],
[0.1103],
[0.0033],
[0.0556],
[0.0096],
[0.0007]], grad_fn=<DivBackward0>)
batch_indices: tensor([534, 380, 28, 669, 687, 974, 452, 65, 883, 233])
l:tensor([[0.0068],
[0.0158],
[0.2469],
[0.0094],
[0.2103],
[0.0059],
[0.3770],
[0.1023],
[0.0935],
[0.1113]], grad_fn=<DivBackward0>)
batch_indices: tensor([325, 330, 484, 424, 526, 58, 793, 898, 41, 766])
l:tensor([[0.0208],
[0.0052],
[0.0704],
[0.1003],
[0.0212],
[0.0096],
[0.1946],
[0.0509],
[0.0055],
[0.0542]], grad_fn=<DivBackward0>)
batch_indices: tensor([770, 820, 677, 435, 708, 530, 605, 144, 959, 764])
l:tensor([[0.1082],
[0.0919],
[0.0614],
[0.0145],
[0.2826],
[0.1062],
[0.2033],
[0.1374],
[0.0543],
[0.0004]], grad_fn=<DivBackward0>)
batch_indices: tensor([439, 899, 173, 847, 857, 697, 654, 696, 941, 313])
l:tensor([[0.0026],
[0.0007],
[0.0003],
[0.2036],

```

```

        [0.1719],
        [0.0661],
        [0.0416],
        [0.0529],
        [0.1257],
        [0.0732]], grad_fn=<DivBackward0>)
batch_indices: tensor([346, 952, 812, 363, 779, 437, 80, 63, 816, 494])
l:tensor([[0.0698],
        [0.0131],
        [0.1961],
        [0.1637],
        [0.1801],
        [0.0661],
        [0.1995],
        [0.0242],
        [0.0879],
        [0.1023]], grad_fn=<DivBackward0>)
batch_indices: tensor([182, 142, 244, 725, 140, 783, 192, 200, 892, 387])
l:tensor([[4.2927e-01],
        [4.0247e-03],
        [2.3935e-02],
        [3.8868e-04],
        [7.4951e-01],
        [8.3338e-03],
        [2.4672e-03],
        [2.6653e-01],
        [5.7514e-02],
        [2.7042e-02]], grad_fn=<DivBackward0>)
batch_indices: tensor([379, 967, 938, 405, 120, 912, 403, 419, 608, 289])
l:tensor([[0.0160],
        [0.0612],
        [0.1693],
        [0.1167],
        [0.0329],
        [0.0261],
        [0.0006],
        [0.1221],
        [0.2049],
        [0.0800]], grad_fn=<DivBackward0>)
batch_indices: tensor([195, 42, 673, 306, 230, 231, 272, 607, 504, 613])
l:tensor([[2.6042e-03],
        [5.0137e-02],
        [7.0764e-04],
        [1.2517e-01],
        [2.7179e-06],
        [5.3022e-02],
        [8.6061e-02],
        [2.0685e-01],
        [5.9039e-02],
        [9.2616e-05]], grad_fn=<DivBackward0>)
batch_indices: tensor([866, 596, 945, 921, 867, 475, 345, 996, 604, 276])
l:tensor([[0.0688],
        [0.0470],
        [0.1290],
        [0.1955],
        [0.0031],

```



```

        [0.0015],
        [0.0775],
        [0.0104],
        [0.0617],
        [0.0510]], grad_fn=<DivBackward0>)
batch_indices: tensor([ 53, 400,  54, 209, 328, 942, 623, 489, 762, 965])
l:tensor([[2.6940e-02],
        [1.5669e-02],
        [6.2722e-03],
        [1.9598e-01],
        [1.1320e-01],
        [2.0613e-02],
        [1.6578e-04],
        [3.5309e-02],
        [1.5179e-02],
        [1.7778e-02]], grad_fn=<DivBackward0>)
batch_indices: tensor([312, 856, 943, 829,  71, 826,  16, 634, 242, 554])
l:tensor([[0.0378],
        [0.0314],
        [0.0244],
        [0.0742],
        [0.2139],
        [0.1609],
        [0.0210],
        [0.2277],
        [0.1053],
        [0.0530]], grad_fn=<DivBackward0>)
batch_indices: tensor([516, 464, 913,  29, 749, 651, 243, 722, 962,  2])
l:tensor([[0.0377],
        [0.0335],
        [0.1141],
        [0.0321],
        [0.0554],
        [0.1618],
        [0.0015],
        [0.0137],
        [0.0731],
        [0.0028]], grad_fn=<DivBackward0>)
batch_indices: tensor([454, 384, 370, 468, 519, 164, 443, 291, 768,  26])
l:tensor([[0.0054],
        [0.0366],
        [0.0593],
        [0.0142],
        [0.0431],
        [0.0275],
        [0.0377],
        [0.0047],
        [0.2929],
        [0.0904]], grad_fn=<DivBackward0>)
batch_indices: tensor([112, 924, 646, 187, 138, 559, 453, 595, 450, 581])
l:tensor([[2.9338e-01],
        [1.8114e-03],
        [1.0421e-04],
        [3.5638e-04],
        [6.5869e-02],
        [8.9377e-03],

```

```

[9.5288e-02],
[5.8883e-02],
[9.7041e-03],
[5.3520e-02]], grad_fn=<DivBackward0>)
batch_indices: tensor([537, 412, 817, 987,  4, 940, 104, 946, 105, 264])
l:tensor([[0.0122],
[0.0003],
[0.0432],
[0.0030],
[0.0471],
[0.0039],
[0.0003],
[0.0518],
[0.0960],
[0.0087]], grad_fn=<DivBackward0>)
batch_indices: tensor([807, 240,  45, 137, 576, 776,  81, 373, 340, 616])
l:tensor([[4.4502e-03],
[2.1408e-01],
[1.2278e-01],
[4.0913e-03],
[3.8159e-03],
[7.7388e-05],
[8.6176e-02],
[1.0151e-01],
[1.0878e-01],
[9.3094e-02]], grad_fn=<DivBackward0>)
batch_indices: tensor([544, 763, 271, 643,  43, 523, 950, 976, 710, 980])
l:tensor([[0.0352],
[0.0656],
[0.0977],
[0.0056],
[0.0745],
[0.1007],
[0.0056],
[0.1416],
[0.0872],
[0.0668]], grad_fn=<DivBackward0>)
batch_indices: tensor([ 5, 814, 286, 672, 944, 752,  0, 907, 873, 828])
l:tensor([[3.0846e-02],
[2.1018e-02],
[1.8665e-02],
[9.2223e-05],
[1.0363e-01],
[9.8076e-03],
[1.6661e-01],
[2.2048e-02],
[5.2831e-03],
[2.4879e-02]], grad_fn=<DivBackward0>)
batch_indices: tensor([383, 309, 799, 715, 948, 930, 876, 794, 393, 839])
l:tensor([[1.0399e-03],
[7.7611e-04],
[3.9873e-06],
[6.9273e-02],
[1.4589e-01],
[3.2766e-04],
[6.0562e-02],

```

```
[2.4500e-03],
[5.0712e-02],
[9.9453e-03]], grad_fn=<DivBackward0>)
batch_indices: tensor([699, 406, 740, 555, 917, 508, 681, 615, 197, 630])
l:tensor([[0.0453],
[0.0294],
[0.0008],
[0.0002],
[0.0009],
[0.0618],
[0.0218],
[0.0005],
[0.0007],
[0.0545]], grad_fn=<DivBackward0>)
batch_indices: tensor([354, 720, 984, 353, 300, 500, 540, 754, 279, 719])
l:tensor([[0.0115],
[0.0607],
[0.0106],
[0.0113],
[0.0049],
[0.0262],
[0.0032],
[0.0299],
[0.0004],
[0.0063]], grad_fn=<DivBackward0>)
batch_indices: tensor([ 92, 746, 421, 563, 507, 682, 514, 741,  64, 590])
l:tensor([[0.0313],
[0.0004],
[0.0697],
[0.0196],
[0.0965],
[0.0210],
[0.2041],
[0.0052],
[0.0131],
[0.0129]], grad_fn=<DivBackward0>)
epoch 1, 平均损失 0.036028
batch_indices: tensor([ 91, 782, 348, 735, 977, 281, 607, 956, 870, 300])
l:tensor([[0.0273],
[0.1305],
[0.0273],
[0.0022],
[0.0193],
[0.0009],
[0.0868],
[0.0348],
[0.0195],
[0.0042]], grad_fn=<DivBackward0>)
batch_indices: tensor([ 73, 263, 629, 446, 295, 104, 566, 415,  10, 389])
l:tensor([[0.0132],
[0.0170],
[0.0504],
[0.0291],
[0.0190],
[0.0002],
[0.0511],
```

```

        [0.0377],
        [0.0700],
        [0.0534]], grad_fn=<DivBackward0>)
batch_indices: tensor([407, 166, 473, 160, 436, 273, 754, 935, 455, 941])
l:tensor([[0.0007],
        [0.1615],
        [0.0463],
        [0.0055],
        [0.0003],
        [0.0094],
        [0.0231],
        [0.2080],
        [0.0183],
        [0.0371]], grad_fn=<DivBackward0>)
batch_indices: tensor([679, 219, 919, 576, 625, 817, 971, 163, 400, 62])
l:tensor([[0.0049],
        [0.0744],
        [0.0466],
        [0.0032],
        [0.0003],
        [0.0200],
        [0.0240],
        [0.0518],
        [0.0071],
        [0.0303]], grad_fn=<DivBackward0>)
batch_indices: tensor([121, 951, 55, 724, 11, 663, 602, 470, 362, 471])
l:tensor([[3.5822e-02],
        [1.0689e-02],
        [1.1883e-02],
        [9.2214e-02],
        [2.5627e-05],
        [1.2752e-04],
        [1.4352e-01],
        [1.8497e-03],
        [2.2856e-02],
        [1.0242e-02]], grad_fn=<DivBackward0>)
batch_indices: tensor([207, 732, 677, 657, 745, 279, 47, 168, 675, 398])
l:tensor([[6.3004e-02],
        [2.8081e-02],
        [1.5011e-02],
        [7.6607e-04],
        [2.1976e-02],
        [1.3358e-05],
        [2.7089e-02],
        [3.9938e-05],
        [8.4409e-02],
        [2.5353e-04]], grad_fn=<DivBackward0>)
batch_indices: tensor([114, 878, 666, 46, 213, 60, 592, 780, 596, 868])
l:tensor([[0.1076],
        [0.1015],
        [0.0018],
        [0.0094],
        [0.0517],
        [0.0267],
        [0.0102],
        [0.0179],

```

```

        [0.0160],
        [0.0384]], grad_fn=<DivBackward0>)
batch_indices: tensor([634, 641, 647, 41, 276, 30, 551, 837, 612, 196])
l:tensor([[0.0675],
        [0.0016],
        [0.0067],
        [0.0013],
        [0.0129],
        [0.0199],
        [0.0653],
        [0.0367],
        [0.1159],
        [0.0117]], grad_fn=<DivBackward0>)
batch_indices: tensor([804, 875, 967, 556, 491, 401, 619, 369, 611, 435])
l:tensor([[0.0540],
        [0.0015],
        [0.0138],
        [0.0022],
        [0.0007],
        [0.0397],
        [0.0772],
        [0.0258],
        [0.0718],
        [0.0021]], grad_fn=<DivBackward0>)
batch_indices: tensor([924, 125, 315, 438, 582, 863, 927, 849, 57, 320])
l:tensor([[0.0010],
        [0.0174],
        [0.0025],
        [0.0183],
        [0.0004],
        [0.0058],
        [0.0004],
        [0.0006],
        [0.0513],
        [0.0049]], grad_fn=<DivBackward0>)
batch_indices: tensor([ 40, 453, 134, 307, 909, 67, 136, 376, 158, 172])
l:tensor([[1.9578e-03],
        [3.1353e-02],
        [1.8331e-02],
        [3.4498e-03],
        [7.2778e-02],
        [1.2667e-02],
        [3.0766e-02],
        [3.0076e-05],
        [4.7643e-02],
        [2.8046e-02]], grad_fn=<DivBackward0>)
batch_indices: tensor([981, 29, 906, 2, 468, 526, 876, 115, 894, 869])
l:tensor([[5.8143e-05],
        [7.3680e-03],
        [1.2192e-02],
        [6.6663e-04],
        [4.5280e-03],
        [2.6026e-03],
        [2.2473e-02],
        [3.0832e-03],
        [8.6442e-02],

```

```

[4.4278e-04]], grad_fn=<DivBackward0>)
batch_indices: tensor([516, 652, 741, 713, 682,  52, 952, 917, 223, 111])
l:tensor([[0.0081],
          [0.0006],
          [0.0029],
          [0.0010],
          [0.0094],
          [0.0077],
          [0.0020],
          [0.0009],
          [0.0103],
          [0.0286]], grad_fn=<DivBackward0>)
batch_indices: tensor([492, 334, 594, 828,  74, 299,  86, 762, 218, 332])
l:tensor([[1.3500e-02],
          [2.7799e-03],
          [6.4325e-06],
          [8.5574e-03],
          [3.9763e-04],
          [1.9381e-03],
          [1.4227e-02],
          [3.0142e-03],
          [5.1137e-03],
          [1.1610e-02]], grad_fn=<DivBackward0>)
batch_indices: tensor([519, 938, 349, 637, 149, 377, 832, 822, 954, 233])
l:tensor([[0.0102],
          [0.0328],
          [0.0084],
          [0.0207],
          [0.0014],
          [0.0149],
          [0.0524],
          [0.0008],
          [0.0020],
          [0.0138]], grad_fn=<DivBackward0>)
batch_indices: tensor([481,  33,  4, 959, 305, 351, 942, 785, 325, 838])
l:tensor([[1.2337e-02],
          [3.8627e-02],
          [1.2004e-02],
          [5.8140e-03],
          [1.2861e-03],
          [1.1345e-02],
          [5.0430e-03],
          [4.4033e-05],
          [2.0908e-03],
          [5.1408e-04]], grad_fn=<DivBackward0>)
batch_indices: tensor([617, 674, 806, 807, 215, 156, 750, 599, 164, 585])
l:tensor([[0.0054],
          [0.0079],
          [0.0007],
          [0.0011],
          [0.0045],
          [0.0285],
          [0.0161],
          [0.0065],
          [0.0078],
          [0.0062]], grad_fn=<DivBackward0>)

```

```

batch_indices: tensor([890, 162, 708, 83, 992, 15, 235, 188, 834, 658])
l:tensor([[0.0003],
          [0.0045],
          [0.0328],
          [0.0107],
          [0.0085],
          [0.0032],
          [0.0115],
          [0.0109],
          [0.0006],
          [0.0013]], grad_fn=<DivBackward0>)
batch_indices: tensor([390, 819, 159, 450, 622, 173, 451, 177, 311, 118])
l:tensor([[2.7436e-02],
          [2.1721e-02],
          [7.5795e-05],
          [1.9036e-03],
          [3.8458e-05],
          [1.0365e-04],
          [2.1401e-02],
          [1.6230e-02],
          [3.5312e-04],
          [4.7009e-02]], grad_fn=<DivBackward0>)
batch_indices: tensor([410, 39, 337, 916, 733, 366, 757, 3, 814, 336])
l:tensor([[0.0102],
          [0.0188],
          [0.0125],
          [0.0209],
          [0.0159],
          [0.0030],
          [0.0011],
          [0.0079],
          [0.0056],
          [0.0159]], grad_fn=<DivBackward0>)
batch_indices: tensor([858, 310, 102, 860, 308, 540, 888, 178, 943, 330])
l:tensor([[9.0867e-05],
          [3.5498e-03],
          [1.3204e-02],
          [4.8665e-04],
          [7.8465e-03],
          [4.0972e-04],
          [6.9007e-03],
          [9.5594e-04],
          [2.7905e-03],
          [3.5585e-04]], grad_fn=<DivBackward0>)
batch_indices: tensor([928, 191, 291, 897, 846, 613, 835, 472, 831, 139])
l:tensor([[3.8011e-04],
          [3.2263e-02],
          [1.2738e-03],
          [1.6775e-02],
          [1.4488e-02],
          [6.4623e-05],
          [2.3349e-04],
          [1.1938e-04],
          [1.2526e-02],
          [9.4255e-06]], grad_fn=<DivBackward0>)
batch_indices: tensor([608, 818, 266, 227, 748, 766, 241, 179, 923, 329])

```

```

1:tensor([[0.0232],
          [0.0002],
          [0.0238],
          [0.0067],
          [0.0028],
          [0.0046],
          [0.0017],
          [0.0028],
          [0.0672],
          [0.0002]], grad_fn=<DivBackward0>)
batch_indices: tensor([776, 45, 813, 189, 209, 440, 615, 873, 397, 671])
1:tensor([[3.5216e-05],
          [2.1031e-02],
          [1.0278e-02],
          [8.1355e-03],
          [2.1586e-02],
          [6.6188e-02],
          [2.3534e-04],
          [8.7774e-04],
          [4.3627e-03],
          [2.3621e-02]], grad_fn=<DivBackward0>)
batch_indices: tensor([228, 240, 338, 122, 692, 84, 518, 541, 132, 892])
1:tensor([[0.0011],
          [0.0338],
          [0.0258],
          [0.0159],
          [0.0057],
          [0.0302],
          [0.0058],
          [0.0071],
          [0.0082],
          [0.0056]], grad_fn=<DivBackward0>)
batch_indices: tensor([532, 454, 35, 769, 957, 524, 313, 176, 405, 497])
1:tensor([[0.0033],
          [0.0022],
          [0.0001],
          [0.0055],
          [0.0057],
          [0.0122],
          [0.0053],
          [0.0210],
          [0.0076],
          [0.0289]], grad_fn=<DivBackward0>)
batch_indices: tensor([844, 683, 141, 380, 801, 18, 699, 181, 49, 805])
1:tensor([[0.0147],
          [0.0584],
          [0.0047],
          [0.0007],
          [0.0215],
          [0.0077],
          [0.0082],
          [0.0033],
          [0.0002],
          [0.0048]], grad_fn=<DivBackward0>)
batch_indices: tensor([ 90, 237, 635, 779, 895, 231, 375, 21, 396, 77])
1:tensor([[0.0011],

```



```

[0.0033],
[0.0200],
[0.0135],
[0.0021],
[0.0048],
[0.0053],
[0.0006],
[0.0081],
[0.0083]], grad_fn=<DivBackward0>)
batch_indices: tensor([355, 787, 695, 272, 742, 383, 200, 823, 747, 874])
l:tensor([[0.0141],
[0.0005],
[0.0077],
[0.0064],
[0.0003],
[0.0005],
[0.0153],
[0.0019],
[0.0009],
[0.0015]], grad_fn=<DivBackward0>)
batch_indices: tensor([504, 773, 445, 696, 1, 891, 560, 256, 249, 244])
l:tensor([[0.0044],
[0.0054],
[0.0092],
[0.0028],
[0.0041],
[0.0210],
[0.0021],
[0.0013],
[0.0114],
[0.0015]], grad_fn=<DivBackward0>)
batch_indices: tensor([717, 458, 793, 734, 183, 628, 116, 193, 587, 17])
l:tensor([[8.5304e-04],
[5.3198e-03],
[4.3450e-03],
[1.2555e-02],
[1.4623e-04],
[4.6205e-03],
[1.5333e-03],
[4.3523e-04],
[7.0053e-05],
[7.3335e-04]], grad_fn=<DivBackward0>)
batch_indices: tensor([245, 206, 24, 578, 531, 797, 982, 714, 488, 129])
l:tensor([[1.4894e-02],
[3.1405e-03],
[9.9957e-04],
[1.6586e-03],
[1.6452e-02],
[4.4755e-03],
[1.4388e-03],
[2.2676e-03],
[1.4028e-04],
[1.4925e-06]], grad_fn=<DivBackward0>)
batch_indices: tensor([881, 42, 187, 6, 350, 414, 501, 494, 146, 510])
l:tensor([[3.7808e-05],
[3.1560e-03],

```

```

[2.4546e-08],
[2.6828e-03],
[2.1113e-03],
[9.9784e-04],
[5.6285e-03],
[5.5271e-03],
[1.4297e-04],
[8.2965e-03]], grad_fn=<DivBackward0>)
batch_indices: tensor([ 53, 359, 953, 408, 643, 120, 460, 498, 727, 925])
l:tensor([[0.0034],
[0.0019],
[0.0005],
[0.0001],
[0.0007],
[0.0025],
[0.0050],
[0.0023],
[0.0009],
[0.0098]], grad_fn=<DivBackward0>)
batch_indices: tensor([493, 562, 885, 331, 486, 262, 7, 965, 391, 546])
l:tensor([[1.5074e-02],
[4.5991e-03],
[6.3874e-08],
[8.2324e-03],
[2.8797e-04],
[4.8074e-03],
[1.3495e-02],
[1.2316e-03],
[3.0066e-03],
[1.0735e-02]], grad_fn=<DivBackward0>)
batch_indices: tensor([ 12, 698, 905, 217, 392, 208, 441, 27, 260, 271])
l:tensor([[1.2459e-03],
[6.1381e-04],
[1.3759e-06],
[2.1036e-03],
[1.6261e-03],
[2.1770e-02],
[2.3825e-03],
[4.3038e-03],
[9.0970e-09],
[8.0811e-03]], grad_fn=<DivBackward0>)
batch_indices: tensor([955, 255, 990, 912, 958, 693, 433, 503, 447, 948])
l:tensor([[1.0805e-03],
[1.4626e-03],
[2.6422e-03],
[9.3794e-04],
[2.0232e-03],
[2.3721e-06],
[7.7008e-04],
[3.0510e-05],
[1.7965e-03],
[1.3856e-02]], grad_fn=<DivBackward0>)
batch_indices: tensor([508, 616, 648, 474, 763, 673, 478, 667, 777, 92])
l:tensor([[0.0047],
[0.0055],
[0.0108],

```

```

        [0.0010],
        [0.0055],
        [0.0003],
        [0.0003],
        [0.0012],
        [0.0004],
        [0.0030]], grad_fn=<DivBackward0>)
batch_indices: tensor([871, 775, 580, 257, 563, 480, 552, 505, 568, 768])
l:tensor([[6.8610e-03],
        [7.9463e-05],
        [1.3532e-10],
        [1.2845e-02],
        [1.0808e-03],
        [1.0387e-03],
        [3.4961e-04],
        [7.2716e-03],
        [2.0493e-03],
        [1.4152e-02]], grad_fn=<DivBackward0>)
batch_indices: tensor([456, 63, 739, 448, 573, 318, 103, 931, 439, 765])
l:tensor([[0.0011],
        [0.0005],
        [0.0070],
        [0.0024],
        [0.0001],
        [0.0002],
        [0.0023],
        [0.0017],
        [0.0002],
        [0.0007]], grad_fn=<DivBackward0>)
batch_indices: tensor([476, 654, 670, 721, 514, 558, 970, 98, 403, 82])
l:tensor([[3.5394e-03],
        [2.1322e-03],
        [9.6850e-03],
        [5.9504e-04],
        [1.5739e-02],
        [2.3795e-04],
        [9.2844e-04],
        [2.1324e-03],
        [3.9540e-05],
        [6.4649e-05]], grad_fn=<DivBackward0>)
batch_indices: tensor([386, 852, 270, 525, 85, 58, 761, 827, 0, 288])
l:tensor([[2.4317e-04],
        [9.1155e-05],
        [1.5629e-03],
        [2.4398e-03],
        [4.3913e-03],
        [2.0765e-04],
        [1.7423e-03],
        [3.8790e-04],
        [9.3366e-03],
        [2.6918e-03]], grad_fn=<DivBackward0>)
batch_indices: tensor([ 48, 409, 557, 113, 443, 247, 851, 69, 979, 778])
l:tensor([[0.0009],
        [0.0023],
        [0.0003],
        [0.0007],

```

```

        [0.0018],
        [0.0021],
        [0.0110],
        [0.0031],
        [0.0015],
        [0.0111]], grad_fn=<DivBackward0>)
batch_indices: tensor([ 78, 904, 988, 151, 712, 314, 887, 633, 825, 381])
l:tensor([[1.3143e-03],
        [1.2360e-03],
        [6.0512e-04],
        [8.3255e-04],
        [2.7941e-03],
        [1.8537e-03],
        [2.1009e-03],
        [4.8307e-03],
        [1.9068e-03],
        [1.8983e-06]], grad_fn=<DivBackward0>)
batch_indices: tensor([171, 328, 991, 880, 918, 655,  88, 718,  75, 239])
l:tensor([[0.0031],
        [0.0032],
        [0.0019],
        [0.0045],
        [0.0005],
        [0.0025],
        [0.0002],
        [0.0028],
        [0.0023],
        [0.0007]], grad_fn=<DivBackward0>)
batch_indices: tensor([756, 947, 210, 465, 542, 112, 812,  22, 205, 802])
l:tensor([[1.6364e-03],
        [8.1249e-03],
        [5.0587e-05],
        [2.2936e-03],
        [2.2695e-04],
        [1.0994e-02],
        [3.9912e-03],
        [2.3393e-05],
        [1.5856e-04],
        [2.1827e-03]], grad_fn=<DivBackward0>)
batch_indices: tensor([250, 199, 264, 746, 290, 324, 197, 142, 709, 659])
l:tensor([[9.5275e-03],
        [5.1230e-03],
        [2.2581e-04],
        [2.1358e-06],
        [5.1279e-03],
        [3.4861e-04],
        [2.8966e-04],
        [7.1175e-05],
        [2.3747e-03],
        [7.1100e-06]], grad_fn=<DivBackward0>)
batch_indices: tensor([294, 583,  36, 140, 789, 333, 306, 509, 222, 364])
l:tensor([[1.5662e-03],
        [7.7799e-05],
        [1.3303e-03],
        [1.5571e-02],
        [5.5897e-04],

```

```

[1.9139e-09],
[2.6985e-03],
[4.2883e-03],
[4.4618e-04],
[1.3247e-03]], grad_fn=<DivBackward0>)
batch_indices: tensor([317, 361, 421, 985, 379, 694, 676, 59, 689, 96])
l:tensor([[1.7160e-03],
[3.6274e-05],
[3.0699e-03],
[8.5923e-04],
[1.9000e-04],
[3.0929e-03],
[1.4491e-03],
[6.0993e-03],
[4.3001e-04],
[1.7216e-03]], grad_fn=<DivBackward0>)
batch_indices: tensor([462, 402, 420, 309, 853, 631, 371, 343, 285, 939])
l:tensor([[1.6868e-03],
[1.2777e-03],
[2.7580e-04],
[4.7075e-05],
[6.2248e-04],
[6.7088e-03],
[6.6542e-04],
[1.1415e-03],
[1.1881e-03],
[2.0327e-05]], grad_fn=<DivBackward0>)
batch_indices: tensor([711, 79, 719, 738, 229, 660, 463, 204, 701, 174])
l:tensor([[0.0051],
[0.0038],
[0.0003],
[0.0057],
[0.0018],
[0.0014],
[0.0017],
[0.0010],
[0.0008],
[0.0011]], grad_fn=<DivBackward0>)
batch_indices: tensor([597, 725, 842, 537, 565, 426, 618, 577, 61, 843])
l:tensor([[5.4046e-04],
[2.1741e-04],
[1.9459e-03],
[2.3668e-04],
[9.8517e-05],
[4.3473e-06],
[1.5640e-03],
[3.5766e-04],
[5.1429e-04],
[4.3017e-06]], grad_fn=<DivBackward0>)
batch_indices: tensor([987, 964, 700, 496, 962, 9, 385, 767, 867, 19])
l:tensor([[1.1252e-05],
[4.5816e-04],
[2.6337e-03],
[4.9265e-03],
[1.4399e-03],
[2.6495e-03],

```

```

[7.5957e-04],
[1.2983e-03],
[3.5675e-05],
[3.7796e-04]], grad_fn=<DivBackward0>)
batch_indices: tensor([649, 715, 723, 44, 117, 840, 131, 487, 704, 547])
l:tensor([[1.1544e-03],
[1.7611e-03],
[3.4715e-04],
[5.0268e-03],
[9.0386e-04],
[5.1316e-03],
[6.0881e-03],
[2.6111e-07],
[1.0467e-04],
[1.2612e-03]], grad_fn=<DivBackward0>)
batch_indices: tensor([821, 352, 14, 38, 601, 394, 809, 598, 570, 564])
l:tensor([[5.2662e-05],
[8.3855e-04],
[2.0562e-05],
[3.3663e-04],
[1.6892e-03],
[4.5432e-04],
[2.2078e-03],
[1.3718e-03],
[2.1869e-03],
[1.2155e-07]], grad_fn=<DivBackward0>)
batch_indices: tensor([344, 378, 908, 192, 153, 395, 621, 123, 529, 961])
l:tensor([[3.9703e-04],
[6.5215e-04],
[3.7861e-04],
[2.9357e-04],
[2.7616e-04],
[4.0250e-03],
[1.7612e-05],
[5.4687e-07],
[5.2212e-04],
[2.1317e-06]], grad_fn=<DivBackward0>)
batch_indices: tensor([656, 246, 109, 586, 258, 774, 944, 630, 848, 796])
l:tensor([[1.2408e-03],
[1.4308e-03],
[1.5110e-03],
[8.1038e-04],
[2.1155e-05],
[6.6169e-04],
[1.9913e-03],
[5.8563e-04],
[3.9412e-03],
[1.9179e-03]], grad_fn=<DivBackward0>)
batch_indices: tensor([753, 859, 603, 994, 899, 555, 224, 506, 624, 781])
l:tensor([[1.3071e-04],
[1.9140e-03],
[2.6749e-03],
[3.0801e-05],
[4.1839e-07],
[2.3940e-07],
[1.8723e-03],

```

```

[1.0837e-04],
[6.6874e-05],
[7.9468e-05]], grad_fn=<DivBackward0>)
batch_indices: tensor([703, 370, 755, 770, 212, 706, 464, 569, 388, 642])
l:tensor([[3.8192e-04],
[7.5952e-04],
[8.4797e-04],
[1.1830e-03],
[1.7875e-04],
[1.7118e-05],
[2.4114e-04],
[8.8590e-05],
[4.3555e-04],
[1.6216e-03]], grad_fn=<DivBackward0>)
batch_indices: tensor([946, 357, 107, 973, 937, 913, 442, 678, 248, 680])
l:tensor([[1.1474e-03],
[3.7567e-04],
[2.3466e-04],
[5.2854e-04],
[8.7320e-04],
[1.4349e-03],
[2.9715e-03],
[1.2858e-04],
[1.9760e-03],
[9.0362e-05]], grad_fn=<DivBackward0>)
batch_indices: tensor([269, 138, 323, 707, 752, 788, 243, 810, 523, 799])
l:tensor([[1.6759e-06],
[6.2058e-04],
[7.4321e-04],
[3.5199e-03],
[3.5818e-05],
[7.9599e-05],
[3.8621e-04],
[3.8354e-04],
[1.5914e-03],
[7.0561e-07]], grad_fn=<DivBackward0>)
batch_indices: tensor([644, 152, 980, 110, 883, 97, 536, 356, 216, 302])
l:tensor([[2.1486e-04],
[1.9002e-04],
[8.5802e-04],
[3.2193e-03],
[6.0181e-04],
[2.3426e-03],
[1.5046e-03],
[5.7318e-04],
[3.8594e-05],
[5.3607e-04]], grad_fn=<DivBackward0>)
batch_indices: tensor([280, 252, 130, 589, 857, 384, 975, 850, 983, 275])
l:tensor([[8.2259e-04],
[1.3387e-04],
[3.9361e-03],
[2.5741e-04],
[1.3087e-03],
[4.7282e-04],
[5.0426e-04],
[1.0074e-04],

```

```

[1.7372e-05],
[1.3140e-03]], grad_fn=<DivBackward0>)
batch_indices: tensor([978, 949, 372, 477, 495, 528, 998, 143, 830, 688])
l:tensor([[2.7042e-04],
[6.3862e-04],
[4.6804e-04],
[4.2489e-05],
[2.2654e-03],
[1.7521e-04],
[4.7256e-04],
[1.1983e-05],
[2.9360e-03],
[3.0236e-04]], grad_fn=<DivBackward0>)
batch_indices: tensor([236, 820, 791, 632, 482, 902, 437, 639, 571, 907])
l:tensor([[1.7981e-03],
[1.0030e-03],
[2.8450e-03],
[1.3778e-03],
[9.6164e-04],
[5.2716e-05],
[2.7761e-04],
[5.6700e-07],
[2.0307e-03],
[5.4189e-04]], grad_fn=<DivBackward0>)
batch_indices: tensor([100, 261, 34, 960, 26, 623, 363, 792, 879, 636])
l:tensor([[3.8940e-08],
[3.4141e-08],
[1.1944e-03],
[2.5002e-04],
[1.1751e-03],
[1.8761e-04],
[1.4382e-03],
[3.6950e-05],
[3.1135e-04],
[4.6540e-03]], grad_fn=<DivBackward0>)
batch_indices: tensor([145, 588, 590, 522, 538, 31, 94, 65, 686, 425])
l:tensor([[3.2370e-04],
[1.0025e-05],
[1.2892e-04],
[3.3282e-03],
[4.0052e-05],
[2.4566e-04],
[1.7782e-03],
[4.2714e-04],
[2.9334e-04],
[2.6627e-04]], grad_fn=<DivBackward0>)
batch_indices: tensor([672, 365, 515, 968, 424, 972, 135, 195, 861, 926])
l:tensor([[1.4657e-04],
[2.6972e-04],
[2.9488e-04],
[8.0486e-05],
[5.4844e-04],
[1.6191e-05],
[3.2902e-04],
[3.1998e-06],
[1.2808e-05],

```



```

[1.9285e-04]], grad_fn=<DivBackward0>)
batch_indices: tensor([662, 934, 382, 99, 512, 150, 884, 896, 404, 428])
l:tensor([[1.2556e-03],
          [6.6027e-04],
          [9.3774e-05],
          [8.1565e-06],
          [3.5440e-04],
          [1.0922e-03],
          [1.2662e-03],
          [2.3711e-04],
          [1.1130e-04],
          [8.7028e-04]], grad_fn=<DivBackward0>)
batch_indices: tensor([743, 685, 502, 427, 251, 640, 889, 669, 292, 772])
l:tensor([[6.3966e-04],
          [2.2636e-04],
          [1.6018e-03],
          [3.4003e-04],
          [2.1536e-04],
          [1.1719e-04],
          [1.3276e-03],
          [1.8167e-05],
          [8.3915e-04],
          [1.9247e-05]], grad_fn=<DivBackward0>)
batch_indices: tensor([457, 232, 815, 903, 722, 71, 684, 23, 559, 882])
l:tensor([[2.7371e-06],
          [1.2368e-03],
          [6.2981e-06],
          [2.5762e-03],
          [1.5070e-04],
          [1.6554e-03],
          [3.9933e-04],
          [6.5277e-08],
          [1.2925e-04],
          [1.3212e-03]], grad_fn=<DivBackward0>)
batch_indices: tensor([ 89, 165, 106, 133, 253, 459, 646, 298, 489, 225])
l:tensor([[1.2327e-03],
          [2.6763e-04],
          [4.0741e-04],
          [3.9641e-05],
          [5.1048e-03],
          [8.6563e-04],
          [3.5513e-05],
          [6.3459e-04],
          [5.3093e-04],
          [2.4809e-03]], grad_fn=<DivBackward0>)
batch_indices: tensor([798, 610, 549, 286, 575, 915, 387, 517, 412, 444])
l:tensor([[3.9192e-04],
          [1.9252e-04],
          [1.0549e-04],
          [1.6092e-04],
          [3.0203e-04],
          [6.1908e-05],
          [2.6083e-05],
          [1.0394e-03],
          [6.5682e-05],
          [2.4248e-04]], grad_fn=<DivBackward0>)

```

```
batch_indices: tensor([816, 345, 731, 936,  50, 108, 749, 167, 429, 475])
l:tensor([[2.5869e-04],
          [5.8149e-04],
          [2.0347e-05],
          [5.8863e-05],
          [2.2849e-04],
          [1.9326e-04],
          [4.7361e-04],
          [7.2967e-04],
          [5.9266e-04],
          [8.3148e-05]]), grad_fn=<DivBackward0>)
batch_indices: tensor([485, 535, 751, 933, 234, 914,  8, 422, 839, 175])
l:tensor([[1.3277e-04],
          [2.1744e-05],
          [3.5080e-05],
          [9.7800e-04],
          [8.6539e-07],
          [1.4512e-03],
          [1.1665e-03],
          [8.6870e-04],
          [2.0220e-04],
          [1.8066e-04]]), grad_fn=<DivBackward0>)
batch_indices: tensor([368, 327, 539,  93, 507, 186, 406, 154, 169, 534])
l:tensor([[8.6287e-06],
          [1.3395e-03],
          [4.2762e-04],
          [4.1030e-06],
          [1.3065e-03],
          [1.6905e-03],
          [3.2522e-04],
          [1.1337e-04],
          [2.2312e-04],
          [1.4182e-05]]), grad_fn=<DivBackward0>)
batch_indices: tensor([866,  13, 341, 373, 339, 811, 418, 862, 744, 691])
l:tensor([[1.9467e-04],
          [5.1757e-04],
          [1.9773e-04],
          [1.9877e-04],
          [6.6419e-04],
          [1.9651e-04],
          [4.0328e-04],
          [8.3186e-04],
          [9.3921e-04],
          [4.1344e-05]]), grad_fn=<DivBackward0>)
batch_indices: tensor([ 95, 729, 561, 974, 449,  16, 854, 877, 543, 984])
l:tensor([[4.9420e-04],
          [6.3528e-05],
          [1.5049e-03],
          [2.9457e-06],
          [1.3982e-04],
          [1.6957e-05],
          [8.6086e-04],
          [3.3216e-05],
          [4.3770e-04],
          [1.0615e-04]]), grad_fn=<DivBackward0>)
batch_indices: tensor([759, 966, 214, 259, 374, 147, 500, 105, 221, 190])
```

```

l:tensor([[1.0500e-03],
          [1.3894e-04],
          [2.0497e-05],
          [3.5076e-07],
          [1.9077e-05],
          [4.1600e-04],
          [7.6710e-05],
          [3.9432e-04],
          [4.4971e-04],
          [2.6412e-05]], grad_fn=<DivBackward0>)
batch_indices: tensor([220, 740, 736, 783, 702, 808, 469,  37, 157, 995])
l:tensor([[2.5541e-04],
          [9.9332e-05],
          [1.8523e-06],
          [2.1942e-04],
          [8.3400e-04],
          [5.5259e-04],
          [1.4771e-06],
          [1.6563e-04],
          [3.7032e-04],
          [8.5812e-06]], grad_fn=<DivBackward0>)
batch_indices: tensor([490, 319, 911, 411, 553, 242, 716, 886, 997, 423])
l:tensor([[8.7945e-06],
          [2.7553e-05],
          [2.7706e-04],
          [4.0264e-05],
          [2.4140e-04],
          [5.6826e-04],
          [1.9899e-05],
          [1.8342e-04],
          [2.9628e-04],
          [5.0107e-05]], grad_fn=<DivBackward0>)
batch_indices: tensor([841, 999, 289, 595,  25, 771, 826, 452,  56, 499])
l:tensor([[1.4471e-03],
          [3.9554e-04],
          [4.5565e-04],
          [6.0575e-05],
          [6.3472e-06],
          [4.2065e-04],
          [4.5582e-04],
          [2.6713e-04],
          [1.2323e-05],
          [4.6683e-04]], grad_fn=<DivBackward0>)
batch_indices: tensor([326, 730, 922, 283, 278, 651, 161, 697, 101, 653])
l:tensor([[2.9974e-05],
          [1.4567e-03],
          [4.0968e-05],
          [7.7304e-06],
          [8.5433e-04],
          [9.1690e-04],
          [7.3774e-04],
          [4.8882e-04],
          [8.0990e-06],
          [7.5922e-06]], grad_fn=<DivBackward0>)
batch_indices: tensor([301, 124, 593, 137, 211, 201,  72,  51, 347, 530])
l:tensor([[3.4904e-04],

```

```

[1.8238e-04],
[4.4799e-04],
[3.3864e-06],
[5.7787e-04],
[2.1305e-05],
[3.5627e-05],
[3.5550e-04],
[5.5531e-05],
[3.9353e-04]], grad_fn=<DivBackward0>)
batch_indices: tensor([550, 335, 910, 627, 930, 898, 230, 544, 969, 836])
l:tensor([[8.7490e-08],
[1.8670e-07],
[1.2320e-05],
[6.7868e-05],
[1.2053e-06],
[1.0602e-04],
[1.6605e-05],
[4.7684e-07],
[6.2077e-04],
[3.8554e-05]], grad_fn=<DivBackward0>)
batch_indices: tensor([726, 198, 119, 367, 304, 786, 28, 893, 996, 600])
l:tensor([[1.4577e-03],
[1.9971e-04],
[6.4792e-05],
[2.8513e-06],
[1.5697e-05],
[2.1668e-04],
[6.8002e-04],
[7.7341e-04],
[9.6288e-05],
[3.5337e-04]], grad_fn=<DivBackward0>)
batch_indices: tensor([681, 127, 432, 321, 668, 128, 993, 184, 545, 360])
l:tensor([[5.5160e-05],
[8.7803e-05],
[5.0538e-05],
[2.7262e-04],
[3.9954e-04],
[2.2641e-05],
[4.0644e-05],
[1.1706e-04],
[4.8758e-05],
[2.5838e-04]], grad_fn=<DivBackward0>)
batch_indices: tensor([ 80, 66, 800, 784, 353, 268, 989, 182, 287, 202])
l:tensor([[2.2314e-05],
[2.5371e-04],
[4.1544e-05],
[1.4520e-03],
[4.6157e-05],
[1.6617e-04],
[3.4102e-04],
[1.2155e-03],
[4.8239e-05],
[3.8094e-04]], grad_fn=<DivBackward0>)
batch_indices: tensor([976, 764, 312, 940, 581, 554, 548, 527, 626, 144])
l:tensor([[2.8431e-04],
[7.1045e-05],

```

```

[7.0904e-05],
[2.9825e-05],
[1.0706e-04],
[3.5032e-05],
[6.5208e-06],
[1.7570e-04],
[1.2170e-04],
[1.9326e-04]], grad_fn=<DivBackward0>)
batch_indices: tensor([170, 354, 479, 194, 606, 645, 614, 609, 579, 690])
l:tensor([[1.1826e-06],
[3.8275e-05],
[1.0932e-04],
[5.1144e-07],
[5.9973e-05],
[2.2283e-04],
[8.8962e-04],
[1.8123e-05],
[6.1308e-06],
[3.4783e-05]], grad_fn=<DivBackward0>)
batch_indices: tensor([ 5, 638, 267, 431, 76, 185, 929, 591, 340, 795])
l:tensor([[2.7117e-04],
[3.2792e-05],
[4.5099e-06],
[9.5360e-05],
[5.1185e-05],
[3.0265e-06],
[3.0117e-05],
[4.9271e-05],
[8.8729e-04],
[2.0977e-05]], grad_fn=<DivBackward0>)
batch_indices: tensor([567, 758, 342, 833, 126, 737, 705, 68, 467, 419])
l:tensor([[1.8777e-04],
[1.4001e-04],
[6.8922e-05],
[4.9517e-04],
[5.2098e-05],
[4.9375e-07],
[3.9371e-04],
[1.1623e-04],
[4.4883e-04],
[4.0836e-04]], grad_fn=<DivBackward0>)
batch_indices: tensor([282, 824, 847, 950, 664, 872, 963, 87, 803, 720])
l:tensor([[2.3224e-04],
[1.4441e-04],
[3.4939e-04],
[3.1972e-07],
[1.1511e-04],
[1.3292e-05],
[5.4252e-04],
[5.3276e-04],
[2.2048e-05],
[9.4467e-05]], grad_fn=<DivBackward0>)
batch_indices: tensor([238, 650, 466, 399, 274, 901, 393, 434, 43, 584])
l:tensor([[1.1794e-04],
[1.4060e-04],
[1.8426e-05],

```

```

[8.3268e-05],
[1.6281e-05],
[5.9619e-09],
[8.5227e-07],
[1.4258e-04],
[1.9236e-04],
[7.6160e-04]], grad_fn=<DivBackward0>)
batch_indices: tensor([430, 574, 865, 760, 794, 277, 513, 945, 358, 920])
l:tensor([[2.0072e-04],
[3.2685e-06],
[4.5769e-06],
[1.6647e-04],
[1.0471e-04],
[9.1821e-06],
[2.4888e-04],
[5.0824e-04],
[7.2235e-05],
[1.9037e-04]], grad_fn=<DivBackward0>)
batch_indices: tensor([ 54,  70, 346, 520,  64, 728, 661, 511, 254, 297])
l:tensor([[3.7751e-06],
[1.2190e-04],
[4.7828e-05],
[6.1140e-05],
[1.8597e-06],
[8.7201e-06],
[3.3608e-05],
[3.5884e-04],
[9.5591e-05],
[8.3807e-05]], grad_fn=<DivBackward0>)
batch_indices: tensor([413, 533, 284, 180, 203, 296, 665, 483, 417,  20])
l:tensor([[5.2427e-05],
[1.0205e-04],
[1.0708e-06],
[3.0536e-05],
[4.9318e-05],
[2.6418e-05],
[7.7742e-05],
[2.3200e-04],
[1.7402e-04],
[1.0418e-04]], grad_fn=<DivBackward0>)
batch_indices: tensor([155, 710, 265, 604, 461, 226, 148, 829, 303,  32])
l:tensor([[3.6495e-04],
[3.2167e-05],
[2.8406e-04],
[1.5491e-05],
[7.4322e-07],
[2.3675e-04],
[2.3057e-06],
[9.0748e-05],
[6.5839e-05],
[9.5248e-05]], grad_fn=<DivBackward0>)
batch_indices: tensor([864, 322, 855, 900, 856, 293, 620, 605,  81, 416])
l:tensor([[1.2142e-04],
[6.8304e-04],
[1.4481e-06],
[7.8427e-07],

```

```

[9.5348e-06],
[1.0377e-04],
[1.8241e-04],
[1.6546e-04],
[3.8707e-04],
[1.6561e-05]], grad_fn=<DivBackward0>)
batch_indices: tensor([484, 845, 921, 316, 790, 687, 521, 986, 572, 932])
l:tensor([[3.3165e-05],
[2.1969e-05],
[1.4860e-04],
[1.2463e-04],
[3.2416e-05],
[3.1759e-04],
[7.1804e-05],
[2.1627e-04],
[2.8874e-04],
[2.9445e-05]], grad_fn=<DivBackward0>)
epoch 2, 平均损失 0.000133
batch_indices: tensor([118, 847, 227, 51, 532, 783, 891, 723, 835, 923])
l:tensor([[5.5974e-04],
[2.0718e-04],
[5.9671e-05],
[1.1443e-04],
[1.9930e-06],
[1.6409e-04],
[1.2147e-04],
[5.1860e-06],
[1.0419e-06],
[9.5156e-04]], grad_fn=<DivBackward0>)
batch_indices: tensor([172, 791, 979, 582, 25, 586, 211, 858, 363, 163])
l:tensor([[1.0156e-04],
[6.4448e-04],
[3.5589e-06],
[7.2280e-05],
[2.5467e-05],
[1.3710e-04],
[3.0947e-04],
[2.5370e-05],
[2.7839e-04],
[2.9751e-05]], grad_fn=<DivBackward0>)
batch_indices: tensor([852, 576, 149, 699, 695, 973, 193, 880, 442, 744])
l:tensor([[3.4114e-07],
[1.3379e-04],
[1.7148e-05],
[3.3348e-05],
[1.8395e-04],
[1.0941e-07],
[3.4125e-06],
[3.2286e-05],
[1.0774e-04],
[2.9379e-04]], grad_fn=<DivBackward0>)
batch_indices: tensor([287, 81, 607, 802, 844, 517, 587, 109, 106, 687])
l:tensor([[4.4908e-05],
[3.1232e-04],
[3.5472e-05],
[3.3130e-04],

```

```

        [6.6712e-05],
        [1.9255e-04],
        [1.7425e-05],
        [1.8407e-04],
        [3.4353e-05],
        [2.5603e-04]], grad_fn=<DivBackward0>)
batch_indices: tensor([ 84, 107, 516,  65, 981, 845, 212, 301, 194,  33])
l:tensor([[1.1993e-06],
        [2.6613e-07],
        [1.2965e-06],
        [1.1768e-05],
        [2.4831e-04],
        [4.7944e-06],
        [1.9513e-05],
        [1.8007e-04],
        [9.9070e-07],
        [2.9260e-04]], grad_fn=<DivBackward0>)
batch_indices: tensor([475, 382, 993, 659, 643, 130, 214, 507, 873, 121])
l:tensor([[4.9372e-05],
        [8.5066e-05],
        [9.8936e-09],
        [1.0189e-04],
        [2.7239e-05],
        [1.6143e-04],
        [1.3819e-06],
        [3.1581e-04],
        [2.5917e-05],
        [9.2701e-06]], grad_fn=<DivBackward0>)
batch_indices: tensor([931, 826, 102, 453, 761, 352, 248, 455, 593, 514])
l:tensor([[1.4322e-05],
        [6.4700e-05],
        [2.5156e-06],
        [4.3669e-05],
        [1.1888e-04],
        [1.2568e-04],
        [4.5903e-05],
        [7.7022e-06],
        [2.0597e-04],
        [3.3308e-04]], grad_fn=<DivBackward0>)
batch_indices: tensor([941, 101,  44, 648, 273, 915,  28, 500, 168, 812])
l:tensor([[3.2620e-05],
        [2.5969e-06],
        [4.7802e-04],
        [2.7258e-04],
        [5.4467e-04],
        [2.0279e-06],
        [2.5893e-04],
        [5.6826e-08],
        [7.4512e-05],
        [4.0593e-05]], grad_fn=<DivBackward0>)
batch_indices: tensor([800, 785, 305,  47, 251, 433, 293, 401, 435, 936])
l:tensor([[4.7414e-06],
        [3.7056e-06],
        [1.2618e-06],
        [1.5006e-05],
        [5.7148e-05],

```



```

[1.5071e-06],
[2.4340e-05],
[6.4119e-06],
[6.3105e-07],
[1.4645e-09]], grad_fn=<DivBackward0>)
batch_indices: tensor([641, 503, 562, 75, 519, 62, 264, 70, 685, 740])
l:tensor([[2.6615e-06],
[2.3786e-06],
[2.8223e-04],
[2.6167e-04],
[2.9854e-05],
[3.3668e-04],
[1.9153e-05],
[6.3014e-05],
[9.4621e-05],
[8.8051e-05]], grad_fn=<DivBackward0>)
batch_indices: tensor([381, 23, 161, 925, 178, 660, 902, 521, 606, 355])
l:tensor([[4.7661e-06],
[1.4698e-05],
[1.8146e-04],
[1.8028e-04],
[1.0743e-06],
[1.2339e-05],
[8.5024e-06],
[5.4354e-05],
[4.0573e-06],
[1.4696e-04]], grad_fn=<DivBackward0>)
batch_indices: tensor([387, 421, 292, 624, 179, 147, 114, 603, 298, 680])
l:tensor([[8.7599e-06],
[4.5270e-05],
[6.4505e-05],
[7.0477e-06],
[9.6126e-05],
[5.5360e-05],
[2.0909e-04],
[2.0121e-04],
[3.9668e-06],
[1.0670e-05]], grad_fn=<DivBackward0>)
batch_indices: tensor([303, 288, 302, 22, 768, 31, 133, 420, 512, 831])
l:tensor([[4.4036e-05],
[2.9465e-05],
[1.9344e-04],
[9.2982e-05],
[5.3229e-05],
[6.3771e-05],
[1.7964e-05],
[7.9847e-05],
[1.7690e-05],
[4.1335e-05]], grad_fn=<DivBackward0>)
batch_indices: tensor([ 27, 868, 775, 662, 875, 764, 299, 934, 138, 185])
l:tensor([[2.1801e-05],
[6.3534e-05],
[4.8643e-05],
[5.4138e-05],
[4.4777e-05],
[7.9468e-05],

```

```

[3.4124e-05],
[9.0114e-05],
[3.6638e-05],
[7.5394e-06]], grad_fn=<DivBackward0>)
batch_indices: tensor([558, 675, 568, 730, 201, 494, 140, 874, 180, 250])
l:tensor([[7.6838e-07],
[2.4025e-04],
[1.7625e-05],
[4.3515e-04],
[8.1086e-05],
[1.5081e-04],
[4.7118e-04],
[8.3277e-05],
[9.0863e-06],
[2.2264e-05]], grad_fn=<DivBackward0>)
batch_indices: tensor([ 35, 901, 825, 939, 942, 726, 628, 279, 839, 997])
l:tensor([[2.2124e-05],
[4.9747e-06],
[4.2776e-06],
[6.9442e-06],
[4.1281e-04],
[6.0763e-04],
[1.0750e-04],
[1.3144e-04],
[7.3078e-05],
[2.2392e-06]], grad_fn=<DivBackward0>)
batch_indices: tensor([671, 645, 510, 816, 549, 160, 450, 862, 164, 518])
l:tensor([[1.6581e-04],
[1.8407e-04],
[3.2538e-04],
[2.4720e-06],
[1.6931e-04],
[7.7427e-05],
[8.6838e-05],
[1.7041e-04],
[2.2088e-04],
[6.6619e-06]], grad_fn=<DivBackward0>)
batch_indices: tensor([ 56, 513, 732, 976, 739, 136, 752, 402, 204,  10])
l:tensor([[2.1932e-06],
[8.2222e-05],
[3.6433e-06],
[1.3716e-05],
[1.2631e-04],
[3.2491e-06],
[6.3000e-05],
[6.9494e-05],
[8.8698e-05],
[7.2448e-05]], grad_fn=<DivBackward0>)
batch_indices: tensor([698, 310, 380, 918, 930, 679, 985, 191, 859, 686])
l:tensor([[8.5445e-05],
[1.3592e-04],
[7.0904e-05],
[3.8089e-05],
[1.2013e-06],
[1.6746e-04],
[3.4968e-06],

```

```

[3.6753e-04],
[1.9324e-04],
[2.3794e-05]], grad_fn=<DivBackward0>)
batch_indices: tensor([871, 378, 76, 638, 347, 307, 684, 911, 887, 423])
l:tensor([[2.1249e-04],
[3.6384e-05],
[8.3273e-06],
[8.7713e-05],
[1.8135e-06],
[3.2364e-06],
[1.2839e-05],
[1.0569e-05],
[1.9304e-04],
[6.6498e-06]], grad_fn=<DivBackward0>)
batch_indices: tensor([781, 830, 295, 654, 872, 408, 108, 386, 112, 249])
l:tensor([[1.6064e-04],
[3.3461e-05],
[3.5411e-04],
[2.4991e-04],
[7.8188e-05],
[1.0915e-04],
[2.2089e-04],
[4.2875e-04],
[1.2171e-04],
[7.4052e-05]], grad_fn=<DivBackward0>)
batch_indices: tensor([972, 1, 523, 384, 19, 632, 465, 896, 702, 459])
l:tensor([[5.2260e-08],
[1.0074e-04],
[1.8072e-05],
[3.9755e-09],
[2.4810e-06],
[1.3128e-04],
[2.7345e-05],
[4.9515e-05],
[6.4624e-05],
[5.0816e-05]], grad_fn=<DivBackward0>)
batch_indices: tensor([854, 358, 285, 555, 598, 540, 600, 539, 738, 737])
l:tensor([[4.4808e-04],
[6.5474e-05],
[2.6453e-05],
[1.6058e-05],
[2.5974e-04],
[9.3167e-05],
[1.0512e-04],
[6.0146e-05],
[1.9809e-04],
[2.9773e-05]], grad_fn=<DivBackward0>)
batch_indices: tensor([478, 262, 559, 0, 935, 226, 531, 829, 529, 473])
l:tensor([[1.3658e-04],
[1.8135e-06],
[1.2936e-05],
[3.7245e-07],
[3.4533e-04],
[3.2095e-05],
[8.9322e-05],
[1.5640e-05],

```

```

        [6.0875e-07],
        [1.3712e-04]], grad_fn=<DivBackward0>)
batch_indices: tensor([205, 962, 449, 278, 970, 327, 469, 491, 318, 729])
l:tensor([[2.9730e-04],
        [3.9840e-07],
        [3.4278e-06],
        [1.7122e-04],
        [1.3712e-05],
        [1.0278e-04],
        [4.6104e-05],
        [7.9806e-07],
        [3.7596e-07],
        [4.2525e-06]], grad_fn=<DivBackward0>)
batch_indices: tensor([787, 489, 365, 80, 152, 396, 9, 921, 247, 282])
l:tensor([[6.4412e-05],
        [1.2513e-04],
        [1.7330e-05],
        [7.7563e-05],
        [1.1953e-06],
        [3.3489e-05],
        [1.0802e-04],
        [1.0728e-05],
        [3.8993e-07],
        [1.0744e-04]], grad_fn=<DivBackward0>)
batch_indices: tensor([673, 548, 484, 171, 135, 626, 208, 272, 949, 577])
l:tensor([[2.3742e-04],
        [2.7946e-05],
        [2.7174e-06],
        [8.8692e-05],
        [1.3060e-05],
        [7.7948e-05],
        [7.7829e-06],
        [1.6469e-05],
        [7.3673e-05],
        [2.1621e-06]], grad_fn=<DivBackward0>)
batch_indices: tensor([424, 68, 260, 944, 498, 888, 122, 928, 487, 464])
l:tensor([[1.2875e-05],
        [1.8755e-05],
        [2.0691e-05],
        [4.5094e-06],
        [4.4281e-06],
        [2.1562e-06],
        [9.6934e-05],
        [1.8419e-04],
        [8.4916e-05],
        [8.5566e-06]], grad_fn=<DivBackward0>)
batch_indices: tensor([926, 537, 678, 927, 280, 470, 746, 572, 515, 988])
l:tensor([[1.8413e-05],
        [1.1402e-05],
        [2.4012e-05],
        [9.1873e-07],
        [6.5129e-05],
        [8.0495e-06],
        [2.6113e-06],
        [7.7932e-05],
        [2.9639e-05],

```

```

[7.1609e-06]], grad_fn=<DivBackward0>)
batch_indices: tensor([ 55, 780,  52, 933, 265, 758, 235, 187, 410, 766])
l:tensor([[1.1041e-05],
          [2.4340e-05],
          [6.4102e-05],
          [3.5569e-05],
          [4.1657e-05],
          [1.0460e-05],
          [1.4111e-05],
          [1.2576e-05],
          [2.6318e-04],
          [5.0472e-06]], grad_fn=<DivBackward0>)
batch_indices: tensor([312, 886, 621, 658, 708, 482, 446, 206, 870, 422])
l:tensor([[8.3448e-06],
          [2.7564e-06],
          [3.8425e-04],
          [5.2085e-05],
          [4.8808e-05],
          [5.1168e-05],
          [4.1709e-06],
          [5.1040e-05],
          [2.0158e-05],
          [1.6572e-06]], grad_fn=<DivBackward0>)
batch_indices: tensor([530, 807, 696, 710, 711,  34,  46,  30, 778, 361])
l:tensor([[8.5327e-05],
          [1.1065e-04],
          [1.7087e-04],
          [8.6307e-06],
          [1.2477e-04],
          [4.1835e-05],
          [1.3686e-04],
          [3.8615e-05],
          [1.6090e-04],
          [1.8813e-04]], grad_fn=<DivBackward0>)
batch_indices: tensor([667, 182, 176, 818,  18, 145, 644, 579, 333, 618])
l:tensor([[3.8774e-06],
          [1.7246e-04],
          [1.1747e-04],
          [1.8521e-05],
          [2.9888e-05],
          [4.5830e-05],
          [4.3518e-07],
          [6.9065e-05],
          [1.7668e-05],
          [1.7729e-05]], grad_fn=<DivBackward0>)
batch_indices: tensor([704, 677, 749, 253, 165, 274, 417, 527, 286, 904])
l:tensor([[7.5375e-08],
          [1.6325e-04],
          [3.0775e-05],
          [4.8415e-04],
          [1.8435e-05],
          [4.0234e-06],
          [6.5527e-05],
          [5.4486e-05],
          [3.7633e-06],
          [7.2101e-05]], grad_fn=<DivBackward0>)

```

```
batch_indices: tensor([827, 544, 615, 760, 233, 841, 219, 210, 430, 861])
l:tensor([[9.4455e-08],
          [1.0612e-04],
          [5.4336e-06],
          [1.0556e-05],
          [2.8248e-05],
          [4.2303e-05],
          [6.7186e-05],
          [1.6792e-06],
          [1.4899e-05],
          [3.1497e-05]], grad_fn=<DivBackward0>)
batch_indices: tensor([741, 588, 60, 917, 550, 599, 857, 439, 919, 569])
l:tensor([[1.2367e-05],
          [8.9905e-06],
          [5.3329e-06],
          [2.1067e-06],
          [3.3996e-07],
          [4.0827e-05],
          [1.5953e-06],
          [7.6254e-06],
          [1.3884e-05],
          [9.7463e-06]], grad_fn=<DivBackward0>)
batch_indices: tensor([849, 938, 94, 372, 284, 611, 951, 63, 296, 770])
l:tensor([[9.2739e-05],
          [3.4209e-04],
          [6.6470e-07],
          [3.4069e-05],
          [3.8687e-06],
          [1.2247e-04],
          [8.9513e-07],
          [7.4847e-05],
          [9.4144e-06],
          [6.7996e-05]], grad_fn=<DivBackward0>)
batch_indices: tensor([371, 755, 334, 379, 38, 348, 485, 404, 268, 203])
l:tensor([[2.3435e-05],
          [7.8983e-05],
          [1.6536e-04],
          [6.5148e-08],
          [1.1602e-06],
          [8.6902e-06],
          [1.2426e-04],
          [3.6221e-06],
          [2.1883e-05],
          [2.7252e-06]], grad_fn=<DivBackward0>)
batch_indices: tensor([311, 93, 747, 575, 243, 843, 245, 457, 7, 882])
l:tensor([[7.5098e-05],
          [9.8403e-06],
          [7.6125e-05],
          [9.9743e-07],
          [1.5722e-04],
          [1.8833e-04],
          [2.2418e-04],
          [1.4083e-05],
          [2.3991e-04],
          [1.5360e-04]], grad_fn=<DivBackward0>)
batch_indices: tensor([215, 49, 567, 614, 222, 565, 335, 3, 552, 534])
```

```

1:tensor([[9.8285e-06],
          [1.5578e-05],
          [7.1438e-05],
          [2.1705e-04],
          [6.1428e-07],
          [3.3758e-05],
          [3.3364e-05],
          [1.0347e-07],
          [8.4357e-06],
          [5.3633e-07]]], grad_fn=<DivBackward0>)
batch_indices: tensor([340, 217, 713, 377, 362, 538, 412, 466, 391, 879])
1:tensor([[2.6736e-04],
          [3.0011e-05],
          [8.1807e-09],
          [2.1651e-07],
          [8.3179e-05],
          [5.6199e-05],
          [1.2246e-04],
          [1.5148e-05],
          [3.4586e-05],
          [8.9513e-05]]], grad_fn=<DivBackward0>)
batch_indices: tensor([427, 894, 810, 218, 629, 789, 159, 137, 705, 325])
1:tensor([[2.8489e-05],
          [1.1722e-05],
          [4.1220e-05],
          [1.6703e-05],
          [5.6361e-06],
          [2.1724e-05],
          [6.1746e-07],
          [4.9616e-06],
          [5.7913e-05],
          [6.1905e-07]]], grad_fn=<DivBackward0>)
batch_indices: tensor([556, 908, 53, 181, 431, 652, 589, 693, 428, 209])
1:tensor([[1.2684e-07],
          [1.9271e-05],
          [2.0239e-04],
          [2.9131e-06],
          [2.4287e-05],
          [1.0119e-04],
          [2.8313e-05],
          [2.0664e-05],
          [9.7526e-05],
          [1.4849e-06]]], grad_fn=<DivBackward0>)
batch_indices: tensor([237, 200, 390, 715, 316, 406, 69, 786, 776, 832])
1:tensor([[5.1269e-05],
          [7.8516e-05],
          [2.2412e-06],
          [6.7914e-06],
          [4.5452e-06],
          [9.8074e-06],
          [4.2699e-07],
          [8.4420e-07],
          [4.5175e-05],
          [4.4931e-07]]], grad_fn=<DivBackward0>)
batch_indices: tensor([866, 150, 438, 326, 436, 192, 26, 83, 683, 111])
1:tensor([[2.8878e-06],

```

```

[8.1320e-05],
[5.2865e-05],
[6.7015e-05],
[4.1094e-05],
[1.3868e-04],
[5.0792e-05],
[6.4657e-05],
[2.3969e-07],
[1.2790e-04]], grad_fn=<DivBackward0>)
batch_indices: tensor([156, 947, 881, 994, 486, 900, 656, 664, 609, 78])
l:tensor([[1.4593e-04],
[3.4191e-05],
[7.8081e-06],
[1.7404e-05],
[4.0900e-04],
[1.7221e-06],
[1.9379e-04],
[4.9958e-06],
[4.0511e-05],
[5.9021e-07]], grad_fn=<DivBackward0>)
batch_indices: tensor([ 39, 501, 623, 186, 613, 856, 590, 197, 946, 905])
l:tensor([[4.0927e-06],
[2.0745e-04],
[2.3043e-04],
[2.8374e-05],
[1.3225e-05],
[9.0248e-05],
[4.6320e-07],
[1.7712e-04],
[1.7101e-04],
[5.4581e-06]], grad_fn=<DivBackward0>)
batch_indices: tensor([980, 440, 146, 407, 99, 98, 256, 29, 89, 992])
l:tensor([[3.9124e-05],
[4.3476e-06],
[5.9462e-07],
[1.1199e-04],
[3.4986e-07],
[2.2817e-06],
[3.0600e-05],
[6.6116e-06],
[6.7186e-05],
[3.6266e-06]], grad_fn=<DivBackward0>)
batch_indices: tensor([631, 706, 511, 445, 753, 493, 50, 694, 999, 476])
l:tensor([[2.0252e-04],
[5.2315e-06],
[5.5929e-07],
[1.8611e-06],
[9.8518e-06],
[2.6485e-04],
[4.0434e-05],
[1.5502e-04],
[1.5565e-05],
[1.9101e-05]], grad_fn=<DivBackward0>)
batch_indices: tensor([604, 773, 591, 836, 297, 502, 701, 907, 244, 90])
l:tensor([[1.5087e-05],
[1.1271e-05],

```



```

[4.0644e-05],
[8.1286e-06],
[3.0694e-04],
[6.5079e-06],
[1.0099e-05],
[4.1145e-05],
[1.2070e-05],
[5.0495e-05]], grad_fn=<DivBackward0>)
batch_indices: tensor([434, 444, 85, 254, 91, 984, 895, 313, 986, 763])
l:tensor([[4.8645e-05],
[4.7605e-06],
[3.9951e-06],
[4.8151e-07],
[2.6297e-05],
[4.5589e-06],
[8.0017e-06],
[9.0051e-10],
[6.9665e-06],
[8.0301e-05]], grad_fn=<DivBackward0>)
batch_indices: tensor([338, 982, 799, 893, 597, 876, 479, 324, 309, 399])
l:tensor([[7.2850e-05],
[7.7237e-06],
[6.5352e-06],
[1.3542e-05],
[3.5541e-05],
[1.5206e-05],
[3.9008e-05],
[8.8962e-05],
[2.7489e-05],
[2.3309e-04]], grad_fn=<DivBackward0>)
batch_indices: tensor([416, 17, 45, 897, 153, 141, 88, 954, 24, 474])
l:tensor([[9.1408e-05],
[6.5686e-05],
[3.9908e-05],
[3.3668e-05],
[5.1337e-09],
[8.0504e-06],
[1.7282e-04],
[1.6126e-07],
[1.0767e-04],
[4.1283e-06]], grad_fn=<DivBackward0>)
batch_indices: tensor([851, 630, 281, 977, 580, 581, 767, 885, 964, 346])
l:tensor([[7.0668e-05],
[2.2703e-04],
[1.0383e-04],
[2.6150e-06],
[1.6732e-05],
[6.1751e-08],
[2.1099e-06],
[1.1459e-05],
[4.1838e-05],
[1.7622e-07]], grad_fn=<DivBackward0>)
batch_indices: tensor([131, 242, 82, 177, 170, 195, 520, 697, 144, 948])
l:tensor([[1.2270e-04],
[3.8649e-05],
[4.9067e-05],

```

```

[2.9418e-04],
[1.9048e-06],
[6.3536e-06],
[6.1018e-05],
[1.4361e-04],
[4.0302e-06],
[1.4962e-04]], grad_fn=<DivBackward0>)
batch_indices: tensor([796, 722, 263, 959, 255, 462, 855, 955, 175, 867])
l:tensor([[2.8868e-05],
[5.6449e-06],
[2.6833e-05],
[2.2966e-05],
[1.6080e-05],
[4.9143e-05],
[6.9331e-05],
[1.3337e-05],
[1.3246e-05],
[7.3222e-06]], grad_fn=<DivBackward0>)
batch_indices: tensor([637, 151, 574, 128, 468, 595, 700, 714, 234, 129])
l:tensor([[6.1768e-06],
[7.7003e-06],
[1.0290e-05],
[2.6871e-05],
[1.7517e-04],
[4.8653e-05],
[1.8458e-05],
[2.2159e-05],
[6.5358e-05],
[5.9526e-05]], grad_fn=<DivBackward0>)
batch_indices: tensor([617, 252, 560, 585, 605, 341, 594, 97, 409, 117])
l:tensor([[4.4786e-05],
[1.6171e-04],
[1.0325e-05],
[1.6957e-05],
[3.0188e-06],
[3.1053e-05],
[1.2987e-04],
[7.0272e-05],
[9.3258e-05],
[1.6623e-05]], grad_fn=<DivBackward0>)
batch_indices: tensor([414, 724, 132, 817, 499, 225, 650, 196, 57, 850])
l:tensor([[1.0937e-04],
[4.1997e-07],
[6.3986e-05],
[2.2721e-05],
[1.1410e-05],
[9.9601e-06],
[1.0118e-04],
[7.9055e-07],
[4.6251e-05],
[5.1870e-07]], grad_fn=<DivBackward0>)
batch_indices: tensor([ 64, 912, 319, 329, 259, 72, 748, 87, 718, 425])
l:tensor([[1.4967e-05],
[1.3695e-05],
[6.3357e-05],
[3.0425e-06],

```

```

[1.6942e-05],
[1.9143e-06],
[1.6745e-05],
[2.4613e-05],
[2.4124e-05],
[5.3332e-05]], grad_fn=<DivBackward0>)
batch_indices: tensor([331, 772, 213, 231, 509, 913, 223, 134, 269, 398])
l:tensor([[5.8959e-05],
[3.5971e-08],
[5.9099e-06],
[4.4000e-05],
[8.2371e-06],
[3.6545e-05],
[8.8761e-06],
[9.5373e-05],
[1.8506e-04],
[3.2451e-05]], grad_fn=<DivBackward0>)
batch_indices: tensor([877, 869, 814, 332, 110, 415, 635, 974, 865, 756])
l:tensor([[1.4611e-06],
[2.9256e-04],
[1.2601e-04],
[1.1280e-05],
[1.8290e-04],
[3.1528e-05],
[1.4194e-04],
[5.4405e-06],
[8.8711e-06],
[7.9760e-06]], grad_fn=<DivBackward0>)
batch_indices: tensor([354, 542, 240, 612, 690, 100, 596, 454, 649, 674])
l:tensor([[1.5358e-06],
[2.3679e-05],
[1.4368e-04],
[1.0522e-04],
[2.2015e-06],
[6.9642e-06],
[3.8268e-05],
[7.1087e-05],
[1.4734e-06],
[2.4815e-06]], grad_fn=<DivBackward0>)
batch_indices: tensor([924, 952, 308, 13, 956, 742, 920, 437, 636, 461])
l:tensor([[1.2021e-04],
[1.7745e-05],
[5.0608e-06],
[4.7233e-05],
[1.2943e-07],
[6.2230e-06],
[2.8501e-06],
[3.0510e-05],
[2.1609e-04],
[1.0217e-06]], grad_fn=<DivBackward0>)
batch_indices: tensor([432, 477, 610, 483, 48, 42, 119, 983, 270, 640])
l:tensor([[9.7863e-06],
[1.0104e-07],
[2.1587e-04],
[1.7261e-05],
[1.0751e-04],

```

```

[4.0573e-06],
[5.6698e-05],
[1.0386e-06],
[1.7343e-05],
[1.0665e-05]], grad_fn=<DivBackward0>)
batch_indices: tensor([471, 543, 190, 848, 601, 966, 592, 12, 728, 224])
l:tensor([[2.6372e-05],
[4.3888e-05],
[9.4048e-05],
[9.9856e-06],
[1.1607e-04],
[2.3881e-05],
[5.3915e-05],
[2.9715e-06],
[1.3592e-05],
[3.2095e-05]], grad_fn=<DivBackward0>)
batch_indices: tensor([707, 963, 343, 991, 375, 257, 809, 546, 202, 716])
l:tensor([[2.8949e-04],
[2.7268e-04],
[6.2679e-05],
[3.1437e-05],
[4.1862e-05],
[1.8825e-04],
[1.3296e-04],
[1.5308e-05],
[1.4033e-05],
[2.8665e-05]], grad_fn=<DivBackward0>)
batch_indices: tensor([166, 526, 492, 359, 668, 784, 754, 634, 762, 735])
l:tensor([[4.7502e-06],
[3.0861e-05],
[1.9080e-05],
[2.3636e-05],
[3.1229e-05],
[9.1527e-05],
[5.6955e-06],
[5.3238e-07],
[4.3546e-06],
[8.8771e-06]], grad_fn=<DivBackward0>)
batch_indices: tensor([751, 92, 36, 771, 945, 61, 143, 289, 797, 120])
l:tensor([[2.2342e-05],
[7.0374e-07],
[8.7432e-05],
[1.7389e-06],
[9.2848e-05],
[5.2645e-05],
[1.5740e-05],
[5.0576e-05],
[6.8676e-05],
[9.0777e-05]], grad_fn=<DivBackward0>)
batch_indices: tensor([725, 103, 584, 395, 443, 602, 884, 666, 169, 291])
l:tensor([[8.5928e-05],
[3.9937e-06],
[1.7801e-04],
[1.3727e-04],
[4.3546e-06],
[3.5675e-04],

```

```

[6.0219e-05],
[1.8063e-07],
[3.7565e-05],
[1.3077e-04]], grad_fn=<DivBackward0>)
batch_indices: tensor([987, 328, 892, 451, 554, 460, 261, 360, 344, 906])
l:tensor([[9.3641e-05],
[7.3873e-06],
[1.2010e-05],
[7.4855e-05],
[3.5577e-05],
[1.0645e-06],
[4.2959e-05],
[4.5307e-05],
[4.0894e-05],
[2.8158e-04]], grad_fn=<DivBackward0>)
batch_indices: tensor([481, 557, 447, 682, 8, 842, 689, 505, 183, 353])
l:tensor([[2.9579e-07],
[6.3608e-06],
[1.2461e-05],
[2.6644e-05],
[1.7560e-04],
[1.5119e-05],
[1.0313e-04],
[1.3490e-04],
[4.5304e-07],
[1.5221e-06]], grad_fn=<DivBackward0>)
batch_indices: tensor([246, 116, 647, 452, 916, 834, 336, 480, 126, 720])
l:tensor([[1.0896e-05],
[3.7223e-06],
[4.0569e-05],
[1.2582e-04],
[5.5450e-05],
[9.9914e-05],
[1.7604e-06],
[2.2065e-05],
[1.9343e-04],
[1.2122e-05]], grad_fn=<DivBackward0>)
batch_indices: tensor([672, 655, 578, 403, 40, 66, 405, 890, 863, 759])
l:tensor([[7.5661e-05],
[6.3975e-05],
[6.3532e-06],
[4.2419e-05],
[9.3950e-05],
[2.4373e-05],
[1.5339e-04],
[2.6998e-05],
[4.7110e-05],
[2.1171e-04]], grad_fn=<DivBackward0>)
batch_indices: tensor([665, 608, 95, 663, 356, 822, 271, 230, 692, 790])
l:tensor([[6.2890e-07],
[5.7044e-09],
[3.2116e-06],
[2.7192e-05],
[4.4936e-06],
[2.1513e-07],
[8.5615e-06],

```

```

[9.8688e-06],
[5.0487e-06],
[7.3751e-07]], grad_fn=<DivBackward0>)
batch_indices: tensor([ 96,  11, 220, 448, 757, 823, 899, 441, 929, 547])
l:tensor([[2.4041e-05],
[1.3013e-04],
[8.1914e-05],
[9.2005e-06],
[1.0848e-05],
[1.6180e-07],
[9.3202e-06],
[2.8320e-05],
[1.1989e-04],
[9.2640e-06]], grad_fn=<DivBackward0>)
batch_indices: tensor([864, 373, 374, 321, 198, 142, 157, 155, 889, 676])
l:tensor([[4.9945e-05],
[9.5466e-05],
[1.7789e-04],
[9.4455e-06],
[2.6919e-06],
[2.5164e-06],
[6.1407e-05],
[3.8603e-05],
[1.4832e-06],
[2.2386e-04]], grad_fn=<DivBackward0>)
batch_indices: tensor([524, 978, 950, 337, 419, 619, 490, 821, 804, 794])
l:tensor([[9.5584e-05],
[4.2431e-05],
[1.0500e-05],
[3.2374e-05],
[5.1729e-05],
[2.8835e-05],
[3.2434e-06],
[6.0924e-06],
[5.2083e-05],
[6.1916e-05]], grad_fn=<DivBackward0>)
batch_indices: tensor([ 67, 276, 385, 625, 801, 188, 323, 294, 456, 646])
l:tensor([[9.9675e-07],
[2.9257e-05],
[4.7270e-05],
[1.8797e-05],
[3.0570e-05],
[2.0676e-05],
[1.0987e-04],
[3.0626e-05],
[4.2106e-07],
[2.3310e-06]], grad_fn=<DivBackward0>)
batch_indices: tensor([392, 467, 397, 342, 779, 691, 661, 522, 566,  4])
l:tensor([[2.7352e-08],
[4.4691e-05],
[1.0956e-05],
[5.8307e-06],
[8.3986e-05],
[1.5013e-06],
[1.5496e-07],
[3.5144e-05],

```

```

[7.0238e-05],
[3.7535e-06]], grad_fn=<DivBackward0>)
batch_indices: tensor([733, 525, 290, 798, 86, 765, 808, 943, 995, 241])
l:tensor([[1.2866e-05],
[9.2537e-05],
[2.3345e-05],
[4.8504e-05],
[2.4266e-08],
[5.2796e-09],
[8.1022e-05],
[3.9507e-05],
[1.8605e-04],
[5.2601e-05]], grad_fn=<DivBackward0>)
batch_indices: tensor([957, 429, 258, 860, 58, 620, 366, 304, 139, 277])
l:tensor([[1.1506e-05],
[2.3215e-07],
[1.8047e-05],
[2.3175e-08],
[1.1282e-05],
[4.9039e-05],
[1.8505e-06],
[4.4254e-05],
[2.7892e-07],
[8.0409e-06]], grad_fn=<DivBackward0>)
batch_indices: tensor([545, 670, 317, 411, 709, 496, 54, 506, 383, 15])
l:tensor([[5.5721e-05],
[6.4180e-07],
[1.5542e-04],
[5.8346e-05],
[5.9592e-05],
[3.4451e-05],
[3.9776e-05],
[1.0192e-04],
[2.5402e-05],
[4.3772e-05]], grad_fn=<DivBackward0>)
batch_indices: tensor([903, 306, 238, 368, 189, 989, 793, 932, 553, 840])
l:tensor([[1.2706e-05],
[6.2188e-06],
[3.8244e-08],
[2.2987e-05],
[2.7574e-04],
[1.7766e-05],
[3.9603e-04],
[5.1538e-06],
[2.1564e-05],
[1.7168e-06]], grad_fn=<DivBackward0>)
batch_indices: tensor([657, 463, 805, 236, 570, 229, 795, 228, 853, 345])
l:tensor([[1.3589e-04],
[2.8036e-06],
[8.0804e-07],
[6.3222e-05],
[4.1327e-05],
[3.7608e-05],
[6.4746e-05],
[8.4347e-06],
[1.5720e-05],

```

```

[8.2661e-06]], grad_fn=<DivBackward0>)
batch_indices: tensor([413, 536, 712, 21, 541, 806, 497, 703, 971, 232])
l:tensor([[3.7398e-06],
[4.3973e-05],
[7.7362e-05],
[5.4042e-05],
[1.7475e-05],
[1.6483e-05],
[4.7025e-05],
[1.3438e-06],
[6.5653e-05],
[5.8262e-04]], grad_fn=<DivBackward0>)
batch_indices: tensor([357, 389, 681, 124, 837, 788, 745, 633, 961, 922])
l:tensor([[4.8022e-05],
[4.1418e-04],
[6.0203e-06],
[6.5545e-06],
[3.5794e-05],
[2.7187e-04],
[1.0572e-04],
[8.2743e-05],
[2.3788e-06],
[8.1770e-05]], grad_fn=<DivBackward0>)
batch_indices: tensor([369, 813, 351, 958, 283, 815, 239, 878, 719, 551])
l:tensor([[8.3205e-06],
[2.9046e-05],
[7.2106e-06],
[8.3575e-06],
[9.2805e-06],
[1.2316e-04],
[1.4418e-04],
[3.9747e-05],
[3.5928e-05],
[8.5177e-05]], grad_fn=<DivBackward0>)
batch_indices: tensor([998, 43, 495, 639, 736, 883, 967, 564, 563, 820])
l:tensor([[7.1282e-07],
[5.0426e-08],
[3.2585e-05],
[2.9788e-06],
[5.1596e-05],
[1.1861e-06],
[1.9272e-05],
[1.9794e-05],
[1.2376e-04],
[6.7430e-05]], grad_fn=<DivBackward0>)
batch_indices: tensor([ 16, 669, 458, 833, 583, 184, 32, 528, 426, 216])
l:tensor([[4.6132e-05],
[1.1773e-04],
[1.3142e-10],
[4.8864e-05],
[5.1999e-05],
[3.1192e-06],
[2.6170e-05],
[3.4112e-05],
[8.1923e-06],
[5.8576e-06]], grad_fn=<DivBackward0>)

```



```

batch_indices: tensor([320, 616, 199, 965, 910, 472, 113, 642, 266, 969])
l:tensor([[1.0510e-07],
          [2.3883e-05],
          [1.1940e-05],
          [1.6810e-07],
          [1.7833e-04],
          [1.4564e-04],
          [9.1729e-06],
          [5.0093e-05],
          [7.6192e-05],
          [1.5732e-04]]], grad_fn=<DivBackward0>)
batch_indices: tensor([148, 743,  2,  59, 769, 105, 330, 315, 651, 838])
l:tensor([[2.2520e-05],
          [3.7262e-06],
          [1.7734e-04],
          [4.9792e-06],
          [4.7090e-05],
          [6.1467e-06],
          [1.7547e-05],
          [5.4417e-06],
          [5.7892e-05],
          [4.2834e-05]]], grad_fn=<DivBackward0>)
batch_indices: tensor([221,  41, 388, 364, 173, 937, 393,  73, 533, 418])
l:tensor([[1.3977e-05],
          [1.6098e-05],
          [1.0392e-05],
          [5.1395e-05],
          [1.5157e-05],
          [3.7674e-05],
          [1.0051e-04],
          [1.8734e-05],
          [8.8982e-06],
          [1.0048e-06]]], grad_fn=<DivBackward0>)
batch_indices: tensor([571, 322, 774, 535, 782, 846, 953, 561,  74, 314])
l:tensor([[7.9475e-07],
          [3.7797e-06],
          [9.8782e-05],
          [1.1553e-04],
          [1.8118e-05],
          [4.6421e-06],
          [3.7307e-06],
          [3.8064e-05],
          [1.2758e-04],
          [9.1824e-07]]], grad_fn=<DivBackward0>)
batch_indices: tensor([750, 488, 777, 162,  5, 968, 349, 622, 400, 300])
l:tensor([[1.7937e-05],
          [8.9083e-07],
          [6.5729e-07],
          [1.6938e-04],
          [3.0139e-05],
          [2.1805e-06],
          [1.5633e-04],
          [2.4288e-05],
          [1.7653e-05],
          [1.9168e-04]]], grad_fn=<DivBackward0>)
batch_indices: tensor([ 37, 125,  77, 940, 819,  79, 828, 367, 167, 792])

```

```

l:tensor([[6.3421e-05],
          [1.3295e-05],
          [1.5504e-05],
          [1.6356e-06],
          [1.0299e-05],
          [9.1708e-05],
          [7.4564e-05],
          [5.1144e-05],
          [1.3998e-04],
          [4.1647e-08]]], grad_fn=<DivBackward0>)
batch_indices: tensor([727, 115, 653, 350, 20, 127, 339, 909, 811, 104])
l:tensor([[1.2038e-06],
          [6.6063e-05],
          [1.4385e-04],
          [1.7846e-05],
          [1.7711e-06],
          [5.4758e-05],
          [1.1912e-06],
          [4.9013e-06],
          [5.2437e-05],
          [1.0061e-04]]], grad_fn=<DivBackward0>)
batch_indices: tensor([627, 207, 990, 803, 960, 914, 158, 996, 731, 508])
l:tensor([[4.7400e-05],
          [6.5320e-06],
          [9.4870e-06],
          [2.0053e-05],
          [4.5467e-06],
          [1.0588e-04],
          [9.9972e-05],
          [2.5812e-05],
          [7.3185e-05],
          [4.8566e-06]]], grad_fn=<DivBackward0>)
batch_indices: tensor([ 71, 898, 376, 721, 688, 154, 734, 14, 975, 504])
l:tensor([[4.4409e-06],
          [1.8450e-06],
          [2.2417e-04],
          [8.7581e-05],
          [2.2127e-04],
          [6.5266e-05],
          [1.1749e-05],
          [1.1171e-05],
          [1.4590e-05],
          [5.1034e-10]]], grad_fn=<DivBackward0>)
batch_indices: tensor([573, 394, 717, 174, 6, 275, 123, 267, 370, 824])
l:tensor([[2.2276e-04],
          [1.0019e-05],
          [4.5194e-06],
          [2.5338e-06],
          [2.7454e-07],
          [6.5976e-07],
          [4.2765e-06],
          [1.2655e-06],
          [2.1591e-05],
          [7.3922e-05]]], grad_fn=<DivBackward0>)
epoch 3, 平均损失 0.000051

```

w 的估计误差: tensor([3.1638e-04, -2.6226e-06], grad_fn=<SubBackward0>)

b 的估计误差: tensor([7.4387e-05], grad_fn=<RsubBackward1>)

4. 矢量化加速

训练模型需要时间，优化算法的效率至关重要。深度学习框架提供的高度优化的**矢量化**

(vectorization) 运算，比手写 `for` 循环快几个数量级。原因：矢量化运算直接调用底层 C++/CUDA 实现的矩阵运算，避免了 Python 解释器的逐元素开销。

下面用 `Timer` 类实测：对两个长度为 10000 的向量做加法，逐元素 `for` 循环 vs 矢量化 `a + b`。

```
In [72]: import math
import time
import numpy as np

class Timer:
    """记录多次运行时间。"""
    def __init__(self):
        self.times = []
        self.start()

    def start(self):
        """启动计时器。"""
        self.tik = time.perf_counter()

    def stop(self):
        """停止计时器并把时间记录到列表中。"""
        self.times.append(time.perf_counter() - self.tik)
        return self.times[-1]

    def avg(self):
        """返回平均时间。"""
        return sum(self.times) / len(self.times)

    def sum(self):
        """返回时间总和。"""
        return sum(self.times)

    def cumsum(self):
        """返回累计时间。"""
        return np.array(self.times).cumsum().tolist()

n = 10000
a = torch.ones(n)
b = torch.ones(n)

# 方式一：for 循环逐元素相加
c = torch.zeros(n)
timer = Timer()
for i in range(n):
    c[i] = a[i] + b[i]
print(f'for 循环: {timer.stop():.5f} 秒')
```

```
# 方式二：矢量化加法
timer.start()
d = a + b
print(f'矢量化: {timer.stop():.5f} 秒')
print(f'加速比约 {timer.times[0] / timer.times[1]:.0f} 倍')
```

for 循环: 0.05385 秒

矢量化: 0.00029 秒

加速比约 188 倍

5. 正态分布与平方损失 —— 为什么用平方损失?

前面我们"理所当然"地使用了平方损失。它为什么是回归问题的合适损失? 本节通过**极大似然估计** (Maximum Likelihood Estimation, MLE) 给出严格的理论依据——这也是本节最重要的推理。

5.1 正态分布

正态分布 (normal distribution, 也叫**高斯分布**, Gaussian distribution) 是统计学中最常见的分布, 其概率密度函数为:

$$p(x) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{1}{2\sigma^2}(x - \mu)^2\right)$$

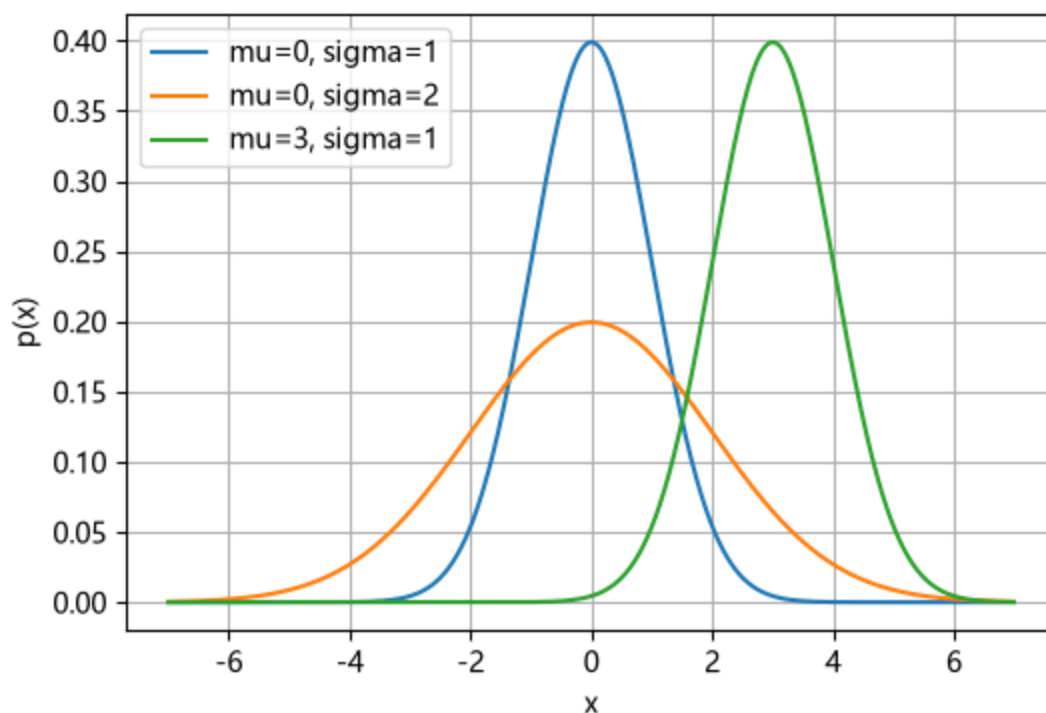
- 均值 μ : 控制分布的**中心位置** (平移曲线) ;
- 方差 σ^2 : 控制分布的**离散程度** (σ 越大曲线越"矮胖", 越小越"高瘦") 。

```
In [73]: import matplotlib.pyplot as plt

def normal(x, mu, sigma):
    """正态分布的概率密度函数。"""
    p = 1 / math.sqrt(2 * math.pi * sigma ** 2)
    return p * np.exp(-0.5 / sigma ** 2 * (x - mu) ** 2)

x = np.arange(-7, 7, 0.01)

# 改变均值产生沿 x 轴的平移, 改变方差则分散或者集中概率密度
params = [(0, 1), (0, 2), (3, 1)]
plt.figure(figsize=(6, 4))
for mu, sigma in params:
    plt.plot(x, normal(x, mu, sigma), label=f'mu={mu}, sigma={sigma}')
plt.xlabel('x')
plt.ylabel('p(x)')
plt.legend()
plt.grid(True)
plt.show()
```



5.2 噪声假设

线性回归假设观测值由真实函数加上噪声得到：

$$y = \mathbf{w}^\top \mathbf{x} + b + \epsilon, \quad \epsilon \sim \mathcal{N}(0, \sigma^2)$$

其中 ϵ 是服从正态分布的噪声。于是，**给定特征 $\mathbf{x}$ ，标签 y 的条件分布也是正态分布**，中心在模型的预测值处：

$$P(y | \mathbf{x}) = \mathcal{N}(y | \mathbf{w}^\top \mathbf{x} + b, \sigma^2) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{1}{2\sigma^2}(y - \mathbf{w}^\top \mathbf{x} - b)^2\right)$$

5.3 极大似然估计的完整推理（重点）

极大似然估计的思想：既然数据已经被观测到了，那就挑选一组参数，使“**这批数据被观测到的概率**”（即似然）最大。

第一步：写出单个样本的似然。

似然 $P(y^{(i)} | \mathbf{x}^{(i)})$ 表示“在参数 $(\mathbf{w}, b, \sigma)$ 给定时，观测到样本 $(\mathbf{x}^{(i)}, y^{(i)})$ 的可能性”，它就是上面的条件密度。

第二步：写出全体样本的联合似然（连乘）。

假设样本 $\{(\mathbf{x}^{(1)}, y^{(1)}), \dots, (\mathbf{x}^{(n)}, y^{(n)})\}$ 是**独立同分布** (i.i.d.) 的，则联合似然是每个样本似然的**乘积**：

$$P(\mathbf{y} | \mathbf{X}) = \prod_{i=1}^n P(y^{(i)} | \mathbf{x}^{(i)}) = \prod_{i=1}^n \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{1}{2\sigma^2}(y^{(i)} - \mathbf{w}^\top \mathbf{x}^{(i)} - b)^2\right)$$

极大似然估计即求解：

$$\operatorname{argmax}_{\mathbf{w}, b, \sigma} P(\mathbf{y} \mid \mathbf{X})$$

第三步：取对数（连乘 → 连加）。

连乘的乘积数值极小（容易数值下溢）且难以求导。由于 $\log$ 是**严格递增函数**，最大化似然与最大化**对数似然**完全等价：

$$\log P(\mathbf{y} \mid \mathbf{X}) = \sum_{i=1}^n \log P\left(y^{(i)} \mid \mathbf{x}^{(i)}\right) = \sum_{i=1}^n \left[-\frac{1}{2} \log(2\pi\sigma^2) - \frac{1}{2\sigma^2} \left(y^{(i)} - \mathbf{w}^\top \mathbf{x}^{(i)} - b \right)^2 \right]$$

第四步：最小化负对数似然。

最大化对数似然 $\iff$ 最小化**负对数似然**（negative log-likelihood）：

$$-\log P(\mathbf{y} \mid \mathbf{X}) = \sum_{i=1}^n \left[\frac{1}{2} \log(2\pi\sigma^2) + \frac{1}{2\sigma^2} \left(y^{(i)} - \mathbf{w}^\top \mathbf{x}^{(i)} - b \right)^2 \right]$$

第五步：识别出平方损失（关键观察）。

现在的目标是关于 $(\mathbf{w}, b)$ 最小化上式。注意：

- 常数项 $\frac{1}{2} \log(2\pi\sigma^2)$ 与 $(\mathbf{w}, b)$ **无关**，不影响最优解；
- 系数 $\frac{1}{2\sigma^2}$ 只是一个**正的常数缩放**，同样不改变关于 $(\mathbf{w}, b)$ 的 argmin 。

于是问题等价于：

$$\operatorname{argmin}_{\mathbf{w}, b} \sum_{i=1}^n \left(y^{(i)} - \mathbf{w}^\top \mathbf{x}^{(i)} - b \right)^2$$

这正是最小化平方误差（均方损失）的目标！

5.4 结论

在高斯噪声假设下，**最小化均方误差 $\iff$ 对线性模型做极大似然估计。**

平方损失并不是随意选择的，而是由“噪声服从正态分布”这一假设，经似然推理**自然导出**的。

补充：若把 σ 也当作待估参数，对它求导可得 $\hat{\sigma}^2 = \frac{1}{n} \sum_i \left(\hat{y}^{(i)} - y^{(i)} \right)^2$ ，
即残差方差——这给出了模型预测的“不确定度”。

5.5 对偶视角：如果噪声是 Laplace 分布？—— L1 损失的由来

同样的推理框架换个噪声假设，会导出不同的损失。假设噪声服从**拉普拉斯分布**（Laplace distribution）：

$$p(\epsilon) = \frac{1}{2\lambda} \exp\left(-\frac{|\epsilon|}{\lambda}\right)$$

写出似然 → 取对数 → 最小化负对数似然，会得到最小化**绝对误差**：

$$\operatorname{argmin}_{\mathbf{w}, b} \sum_{i=1}^n \left| y^{(i)} - \mathbf{w}^\top \mathbf{x}^{(i)} - b \right|$$

这就是 **L1 损失**（平均绝对误差 MAE）的理论来源。

噪声假设	由负对数似然导出的损失
正态（高斯） $\mathcal{N}(0, \sigma^2)$	平方损失（L2 / MSE）
拉普拉斯 Laplace(0, λ)	绝对值损失（L1 / MAE）

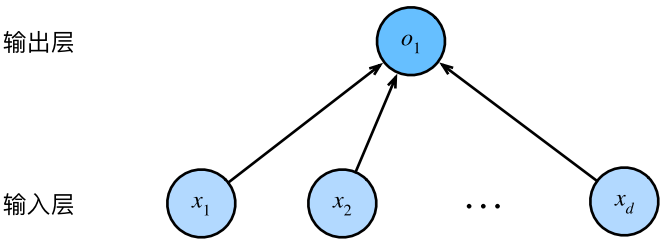
损失函数的选择 = 对噪声分布的假设，这是连接"统计模型"与"优化目标"的核心桥梁。

6. 从线性回归到深度网络

虽然神经网络能表达比线性模型丰富得多的函数，但我们仍可以用描述神经网络的方式描述线性模型——把它看作一个"最简单"的神经网络，从而建立从线性模型到深度网络的桥梁。

6.1 神经网络图示

把线性回归模型画成神经网络：



- **输入**：样本特征 $x_1, \dots, x_d$ （图中 $d = 3$ ），输入层的神经元数即输入维度；
- **输出**： $\mathbf{w}^\top \mathbf{x} + b$ 的计算结果 o_1 ，输出层的神经元数即输出维度；
- 图中只显示**连接模式**（每个输入与每个输出相连），省略了权重和偏置的具体取值；
- 由于输入层不涉及任何计算，按照惯例（输入层不计层数），线性回归是一个**单层神经网络**；

- 输出数为 1 时是单变量回归；若要同时预测多个值（如房价、面积、单价），则是**多变量回归**。

6.2 全连接层

对于线性回归，**每个输入都与每个输出相连**（本例只有一个输出）：

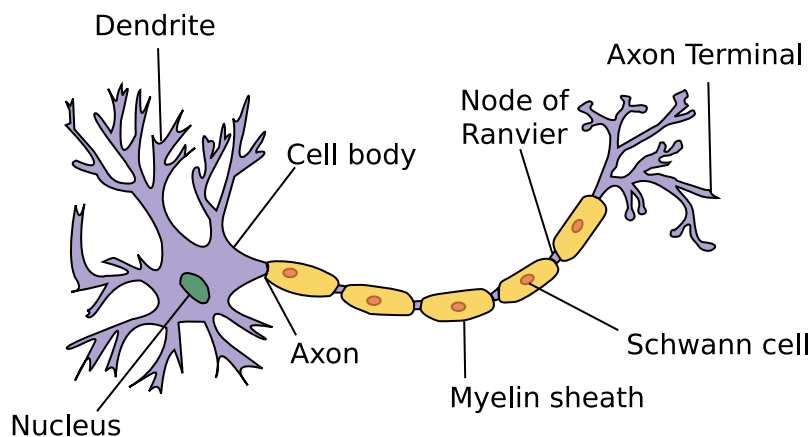
$$o_1 = w_{11}x_1 + w_{12}x_2 + w_{13}x_3 + b_1$$

这种变换称为**全连接层**（fully-connected layer）或**稠密层**（dense layer）。后续章节会大量使用它。

也存在无法用这种“标准神经网络层”描述的模型（如决策树），它们同样有输入与输出，只是计算方式不同。

6.3 生物学上的合理性（可选阅读）

线性回归最早由高斯于 1795 年提出（最小二乘法）。而“人工神经元”的思想，来自对**生物神经元**的粗糙类比：



- **树突**（dendrite）：接收来自其他神经元的电信号 —— 对应输入 x_i ；
- **轴突**（axon）：向其他神经元输出信号 —— 对应输出 o ；
- **突触**（synapse）：神经元之间的连接，信号强度可变 —— 对应权重 w_i ；
- 当输入信号的加权强度超过某个阈值时，神经元被激活 —— 类似偏置 b 与激活函数的思想。

两者思想的共同点在于“**信息的加权聚合**”，但真实生物神经元的复杂程度远超这一简单模型。

线性回归的简洁实现

生成数据集


```
In [74]: import numpy as np
import torch
from torch.utils import data
```

```
In [75]: true_w = torch.tensor([2, -3.4])
true_b = 4.2
features, labels = synthetic_data(true_w, true_b, 1000)
print('features shape:', features.shape)
```

features shape: torch.Size([1000, 2])

读取数据集

```
In [76]: def load_array(data_arrays, batch_size, is_train=True):
        """构造一个Pytorch数据迭代器"""
        dataset = data.TensorDataset(*data_arrays)
        return data.DataLoader(dataset, batch_size, shuffle=is_train)
```

```
In [77]: batch_size = 10
data_iter = load_array((features, labels), batch_size)
next(iter(data_iter))
```

```
Out[77]: [tensor([[ 0.5307, -0.5843],
                  [-1.1204, -0.2059],
                  [ 0.4579,  1.4731],
                  [ 1.6695,  2.1238],
                  [-0.8354,  0.2397],
                  [ 0.7999, -1.3805],
                  [-0.0954,  1.6460],
                  [-0.3967,  0.1929],
                  [ 0.7101,  0.3981],
                  [ 0.0025,  0.0333]]),
          tensor([[ 7.2416],
                  [ 2.6638],
                  [ 0.1069],
                  [ 0.3122],
                  [ 1.7076],
                  [10.4883],
                  [-1.5802],
                  [ 2.7611],
                  [ 4.2720],
                  [ 4.0917]])]
```

定义模型

```
In [78]: from torch import nn

net = nn.Sequential(nn.Linear(2, 1))
```

定义损失函数

```
In [79]: loss=nn.MSELoss()
```

定义优化算法

```
In [80]: trainer=torch.optim.SGD(net.parameters(),lr=0.03)
print(net.parameters())
```

<generator object Module.parameters at 0x000001B54BC96420>

训练

```
In [81]: num_epochs=3

for eopch in range(num_epochs):
    for X,y in data_iter:
        l=loss(net(X),y)
        trainer.zero_grad()
        l.backward()
        trainer.step()
    print(f"epoch {eopch}, loss {l.item():.4f}")
```

epoch 0, loss 0.0002

epoch 1, loss 0.0001

epoch 2, loss 0.0001

```
In [51]: w=net[0].weight.data
print('w的估计误差:',true_w-w.reshape(true_w.shape))
b=net[0].bias.data
print('b的估计误差:',true_b-b)
```

w的估计误差: tensor([-0.0003, -0.0006])

b的估计误差: tensor([0.0007])

3.4 softmax回归

参考：《动手学深度学习》[3.4 softmax回归](#)

[3.1 节](#)的线性回归用于预测多少的问题（房价、住院天数等）。但我们同样关心分类问题——不是问“多少”，而是问“哪一个”：

- 某个电子邮件是否属于垃圾邮件？
- 某个用户会注册还是不注册订阅服务？
- 某个图像描绘的是驴、狗、猫还是鸡？
- 某人接下来最有可能看哪部电影？

"分类"这个词实际描述了两个微妙差别的问题：

1. **硬性类别**：只关心样本属于哪个类别；
2. **软性类别**：希望得到属于每个类别的**概率**。

两者的界限往往很模糊：即使只关心硬类别，我们通常也使用软性（输出概率）的模型。**softmax 回归**就是分类问题的“线性回归”——它也是一个线性模型 + 单层神经网络。

1. 分类问题

从一个图像分类问题开始：假设输入是 2×2 的灰度图像，每个像素用一个标量表示，则每张图像对应 4 个特征 x_1, x_2, x_3, x_4 ；假设每张图像属于“猫”“鸡”“狗”三类之一。

如何表示标签？ 最直接的想法是 $y \in \{1, 2, 3\}$ （分别代表狗、猫、鸡）。如果类别之间有**自然顺序**（如 {婴儿, 儿童, 青少年, 青年人, 中年人, 老年人}），把它当回归问题处理是合理的。

但一般分类问题的类别之间**没有自然顺序**——“猫”和“鸡”之间没有大小关系，直接用整数标签会引入虚假的距离关系。统计学家早就发明了表示分类数据的方法：**独热编码**（one-hot encoding）——一个分量数与类别数相同的向量，对应类别的分量设为 1，其余全为 0：

$$y \in \{(1, 0, 0), (0, 1, 0), (0, 0, 1)\}$$

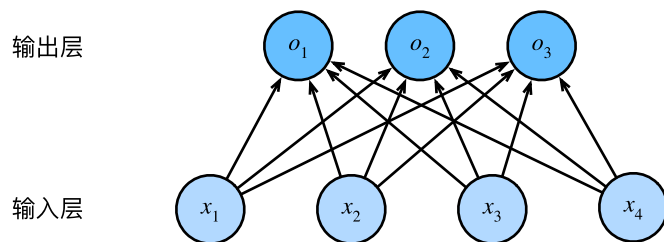
其中 $(1, 0, 0)$ 对应“猫”、 $(0, 1, 0)$ 对应“鸡”、 $(0, 0, 1)$ 对应“狗”。

2. 网络架构

为了估计**所有可能类别的条件概率**，模型需要有多输出——每个类别对应一个输出。对于 4 个特征、3 个类别的例子，每个输出对应自己的**仿射函数**，需要 12 个权重标量 (w_{ij}) 和 3 个偏置标量 (b_i)。对每个输入计算 3 个**未规范化的预测**（unnormalized prediction，也叫 **logit**） o_1, o_2, o_3 ：

$$\begin{aligned} o_1 &= x_1 w_{11} + x_2 w_{12} + x_3 w_{13} + x_4 w_{14} + b_1, \\ o_2 &= x_1 w_{21} + x_2 w_{22} + x_3 w_{23} + x_4 w_{24} + b_2, \\ o_3 &= x_1 w_{31} + x_2 w_{32} + x_3 w_{33} + x_4 w_{34} + b_3. \end{aligned}$$

用神经网络图描述：



与线性回归一样，**softmax 回归也是一个单层神经网络**。由于每个输出 o_1, o_2, o_3 都取决于所有输入 x_1, x_2, x_3, x_4 ，其输出层也是**全连接层**。

用线性代数符号简洁表达为**向量形式**：

$$\mathbf{o} = \mathbf{W}\mathbf{x} + \mathbf{b}$$

所有权重放进一个 3×4 的矩阵 $\mathbf{W}$ ， $\mathbf{b} \in \mathbb{R}^3$ 是偏置向量。

3. 全连接层的参数开销

深度学习中全连接层无处不在，但"完全连接"意味着可能有很多可学习参数。对于具有 d 个输入和 q 个输出的全连接层，参数开销为：

$$\mathcal{O}(dq)$$

在实践中这可能高得令人望而却步（例如 $d = 1000$ 、 $q = 10000$ 时就是一千万参数）。幸运的是，将 d 个输入转换为 q 个输出的成本可以压缩到

$$\mathcal{O}\left(\frac{dq}{n}\right)$$

其中超参数 n 由我们灵活指定，以在实际应用中**平衡参数节约和模型有效性**（这正是后续章节中"隐藏层/低秩分解/注意力机制"等技术的动机之一）。

4. softmax 运算

现在希望把输出 $\hat{y}_j$ 视为"属于类 j 的概率"，然后选择最大输出值的类别 $\operatorname{argmax}_j \hat{y}_j$ 作为预测。例如 $\hat{y}_1, \hat{y}_2, \hat{y}_3 = 0.1, 0.8, 0.1$ 时，预测类别是 2 ("鸡")。

能否把未规范化的预测 $\mathbf{o}$ 直接当概率？ 不能。直接把线性层输出视为概率有两个问题：

1. 输出的总和**不保证为 1**；
2. 输出**可能为负**。

这两点都违反概率的基本公理。要将输出视为概率，必须保证输出**非负且总和为 1**；此外还需要一个训练目标来激励模型精准地估计概率（即校准，calibration：在预测概率为 0.5 的样本中，实际上应约有一半属于该类别）。

softmax 函数（卢斯于 1959 年在选择模型理论中提出）同时满足这些要求：先对每个未规范化的预测**求幂**（保证非负），再除以总和（保证和为 1）：

$$\hat{\mathbf{y}} = \operatorname{softmax}(\mathbf{o}), \quad \hat{y}_j = \frac{\exp(o_j)}{\sum_k \exp(o_k)}$$

softmax 的性质：

- 对所有 j 有 $0 \leq \hat{y}_j \leq 1$ ，且 $\sum_j \hat{y}_j = 1$ ，因此 $\hat{\mathbf{y}}$ 是一个**合法的概率分布**；
- softmax 运算**不改变 $\mathbf{o}$ 之间的大小次序**，只确定分配给每个类别的概率，因此预测类别可以直接写为：

$$\operatorname{argmax}_j \hat{y}_j = \operatorname{argmax}_j o_j$$

- 尽管 softmax 本身是非线性函数，但 softmax 回归的输出仍由输入特征的**仿射变换**决定——所以 softmax 回归依然是一个**线性模型**。

```
In [82]: import torch

def softmax(o):
    """softmax 运算：将未规范化的预测 o（每行一个样本）变换为概率。"""
    o_exp = torch.exp(o)
    partition = o_exp.sum(1, keepdims=True) # 每行求分母
    return o_exp / partition                # 广播机制

o = torch.tensor([[0.5, 2.0, -1.0],
                  [-2.0, -3.0, -2.5]])
y_hat = softmax(o)
print('未规范化的预测 o:\n', o)
print('softmax 后的概率 y_hat:\n', y_hat.numpy().round(4))
print('每行概率之和:', y_hat.sum(1).numpy())
# softmax 不改变大小次序 -> 预测类别 = argmax o
print('argmax o      :', o.argmax(1).numpy())
print('argmax y_hat:', y_hat.argmax(1).numpy())
```

```
未规范化的预测 o:
tensor([[ 0.5000,  2.0000, -1.0000],
        [-2.0000, -3.0000, -2.5000]])
softmax 后的概率 y_hat:
[[0.1753 0.7856 0.0391]
 [0.5065 0.1863 0.3072]]
每行概率之和: [1.          1.0000001]
argmax o      : [1 0]
argmax y_hat: [1 0]
```

5. 小批量样本的矢量化

为了提高计算效率并充分利用 GPU，通常对小批量样本执行矢量计算。设批量大小为 n 、特征维度为 d 、类别数为 q ，则：

$$\mathbf{X} \in \mathbb{R}^{n \times d}, \quad \mathbf{W} \in \mathbb{R}^{d \times q}, \quad \mathbf{b} \in \mathbb{R}^{1 \times q}$$

softmax 回归的矢量计算表达式为：

$$\mathbf{O} = \mathbf{XW} + \mathbf{b},$$

$$\hat{\mathbf{Y}} = \text{softmax}(\mathbf{O}).$$

- $\mathbf{XW} + \mathbf{b}$ 中的加法使用**广播机制**（ $\mathbf{b}$ 加到每一行）；
- $\mathbf{X}$ 的每一行代表一个样本，因此 softmax **按行**（rowwise）执行：对 $\mathbf{O}$ 的每一行先求幂、再归一化；
- 未规范化的预测 $\mathbf{O}$ 和输出概率 $\hat{\mathbf{Y}}$ 都是形状为 $n \times q$ 的矩阵。

6. 损失函数：对数似然与交叉熵

接下来需要损失函数来度量预测效果。方法与线性回归完全相同：**极大似然估计**（回顾 3.1 节的**似然推理**：写似然 $\rightarrow$ 取对数 $\rightarrow$ 最小化负对数似然）。

6.1 对数似然

softmax 给出的向量 $\hat{\mathbf{y}}$ 可视为"给定输入 $\mathbf{x}$ 时每个类别的条件概率", 例如

$$\hat{y}_1 = P(y = \text{猫} \mid \mathbf{x}).$$

设数据集 $\{\mathbf{X}, \mathbf{Y}\}$ 有 n 个样本, 样本 i 由特征向量 $\mathbf{x}^{(i)}$ 和独热标签向量 $\mathbf{y}^{(i)}$ 组成。假设样本独立, 则似然为:

$$P(\mathbf{Y} \mid \mathbf{X}) = \prod_{i=1}^n P(\mathbf{y}^{(i)} \mid \mathbf{x}^{(i)})$$

根据极大似然估计, 最大化 $P(\mathbf{Y} \mid \mathbf{X})$ 等价于**最小化负对数似然**:

$$-\log P(\mathbf{Y} \mid \mathbf{X}) = \sum_{i=1}^n -\log P(\mathbf{y}^{(i)} \mid \mathbf{x}^{(i)}) = \sum_{i=1}^n l(\mathbf{y}^{(i)}, \hat{\mathbf{y}}^{(i)})$$

其中单个样本的损失函数为:

$$l(\mathbf{y}, \hat{\mathbf{y}}) = -\sum_{j=1}^q y_j \log \hat{y}_j$$

由于 $\mathbf{y}$ 是长度为 q 的**独热编码**向量, 除真实类别对应的一项外, 其他所有项都消失, 损失实际就是 $-\log \hat{y}_{\text{真实类别}}$ 。又因为所有 $\hat{y}_j$ 都是概率 ($\log \hat{y}_j \leq 0$), 若能完美预测 ($P(y \mid \mathbf{x}) = 1$), 损失达到最小值 0。注意这通常不可能: 数据集可能存在**标签噪声** (样本被误标), 或输入特征本身信息不足。

6.2 softmax 及其导数

softmax 与交叉熵的组合有一个极为优雅的性质。把 softmax 定义代入损失:

$$\begin{aligned} l(\mathbf{y}, \hat{\mathbf{y}}) &= -\sum_{j=1}^q y_j \log \frac{\exp(o_j)}{\sum_{k=1}^q \exp(o_k)} \\ &= \sum_{j=1}^q y_j \log \sum_{k=1}^q \exp(o_k) - \sum_{j=1}^q y_j o_j \\ &= \log \sum_{k=1}^q \exp(o_k) - \sum_{j=1}^q y_j o_j. \end{aligned}$$

(第三步用了 $\sum_j y_j = 1$, 因为 $\mathbf{y}$ 是独热向量。)

对任意未规范化的预测 o_j 求导:

$$\partial_{o_j} l(\mathbf{y}, \hat{\mathbf{y}}) = \frac{\exp(o_j)}{\sum_{k=1}^q \exp(o_k)} - y_j = \text{softmax}(\mathbf{o})_j - y_j$$

解读：梯度就是"模型分配的概率"与"实际情况（独热标签）"之间的**差异**。这与线性回归中"梯度是 $\hat{y} - y$ "完全同构，并非巧合——在任何**指数族分布**模型中，对数似然的梯度都是这种形式。这使得梯度计算在实践中非常容易。

```
In [83]: # 验证：交叉熵关于 logit 的梯度 = softmax(o) - y (独热标签)
def cross_entropy(y_hat, y_idx):
    """交叉熵损失：取真实类别对应预测概率的负对数。"""
    return -torch.log(y_hat[range(len(y_hat)), y_idx])

o = torch.tensor([0.5, 2.0, -1.0], requires_grad=True) # 3 个 logit
y_idx = 1 # 真实类别
y_onehot = torch.eye(3)[y_idx]
print('y_onehot:', y_onehot)

l = cross_entropy(torch.softmax(o, dim=0), torch.tensor([y_idx])).sum()
l.backward()

print('交叉熵损失      :', l.item())
print('自动微分梯度   :', o.grad.numpy().round(6))
print('softmax(o)-y   :', (torch.softmax(o, dim=0).detach().squeeze() - y_onehot).num

y_onehot: tensor([0., 1., 0.])
交叉熵损失      : 0.24131131172180176
自动微分梯度   : [ 0.17529 -0.214403  0.039113]
softmax(o)-y   : [ 0.17529 -0.214403  0.039113]
```

6.3 交叉熵损失

现在考虑整个**结果分布**的情况：标签不再只是独热向量 $(0, 0, 1)$ ，而可以是概率向量，如 $(0.1, 0.2, 0.7)$ （软标签）。此时损失

$$l(\mathbf{y}, \hat{\mathbf{y}}) = - \sum_{j=1}^q y_j \log \hat{y}_j$$

是"所有标签分布的预期损失"，称为**交叉熵损失**（cross-entropy loss）——分类问题最常用的损失之一。为什么叫"交叉熵"？下一节用信息论解释。

7. 信息论基础

信息论（information theory）研究编码、解码、发送以及尽可能简洁地处理信息。这里只需要其中几个概念，就能完全理解交叉熵。

7.1 信息量

信息论的核心思想是**量化数据中的信息内容**。香农提出用**信息量**（自信息）来量化"惊异程度"：

$$\log \frac{1}{P(j)} = -\log P(j)$$

- 一个事件发生的概率越低，观察它时我们越"惊异"，信息量越大；
- 直觉例子：数据流中每个数据都完全相同 → 我们总能预测下一个数据 → "下一个数据是xx"这个事件毫无信息量。

7.2 熵

分布 P 的熵 (entropy) 是当分配的概率真正匹配数据生成过程时，**信息量的期望**：

$$H[P] = \sum_j -P(j) \log P(j)$$

信息论基本定理之一：为了对从分布 P 中随机抽取的数据进行编码，**至少需要 $H[P]$ "纳特" (nat)**。纳特相当于比特 (bit)，只是对数底为 e 而不是 2 ($1 \text{ 纳特} = \frac{1}{\log 2} \approx 1.44 \text{ 比特}$)。

7.3 重新审视交叉熵

如果把熵 $H(P)$ 想象为"知道真实概率的人所经历的惊异程度"，那么**交叉熵** (从 P 到 Q ，记为 $H(P, Q)$) 就是：

主观概率为 Q 的观察者，在看到根据概率 P 生成的数据时的预期惊异。

$$H(P, Q) = \sum_j -P(j) \log Q(j)$$

关键性质：当 $P = Q$ 时交叉熵达到最低，此时 $H(P, P) = H(P)$ 。

简而言之，分类目标可以从两方面等价理解：

1. **最大化观测数据的似然** (统计视角)；
2. **最小化传达标签所需的惊异** (信息论视角) —— 用模型分布 $\hat{y}$ 编码真实标签 y 。

```
In [84]: # 熵与交叉熵的数值演示: P = Q 时交叉熵最小, 等于熵
def entropy(p):
    """熵 H(P)。"""
    return -(p * p.log()).sum()

def cross_entropy_pq(p, q):
    """交叉熵 H(P, Q)。"""
    return -(p * q.log()).sum()

p = torch.tensor([0.1, 0.2, 0.7]) # 真实分布 P
q_uniform = torch.full((3,), 1 / 3) # 均匀分布
q_close = torch.tensor([0.15, 0.25, 0.6]) # 靠近 P 的分布

print(f'H(P)           = {entropy(p):.4f}')
print(f'H(P, P)        = {cross_entropy_pq(p, p):.4f}    <- P=Q: 交叉熵 = 熵 (最小)')
print(f'H(P, Q近)      = {cross_entropy_pq(p, q_close):.4f}')
print(f'H(P, Q均匀)    = {cross_entropy_pq(p, q_uniform):.4f}')
```


$H(P) = 0.8018$
 $H(P, P) = 0.8018 \quad \leftarrow P=Q: \text{交叉熵} = \text{熵 (最小)}$
 $H(P, Q_{\text{近}}) = 0.8245$
 $H(P, Q_{\text{均匀}}) = 1.0986$

8. 模型预测和评估

训练好 softmax 回归模型后，对任何样本特征都可以预测每个类别的概率；通常取**预测概率最高的类别**作为输出类别。如果预测与实际标签一致，则预测正确。

评估模型用**精度**（accuracy）：正确预测数与预测总数之比：

$$\text{精度} = \frac{\text{正确预测数}}{\text{预测总数}}$$

后续实验中将用精度评估分类模型的性能。

3.7 softmax回归的简洁实现

在 3.3 节（线性回归的简洁实现）中，我们发现通过深度学习框架的高级 API 能够使实现线性回归变得更加容易。同样，通过深度学习框架的高级 API 也能更方便地实现 softmax 回归模型。

本节继续使用 **Fashion-MNIST 数据集**（d2l 在 3.5 节专门介绍），并保持批量大小为 256。与 softmax 回归的从零开始实现（3.6 节）相比，高级 API 让我们只需要关注：

步骤	简洁实现方式
模型	<code>nn.Sequential(nn.Flatten(), nn.Linear(784, 10))</code>
损失	<code>nn.CrossEntropyLoss</code> （内置 LogSumExp 数值稳定技巧）
优化	<code>torch.optim.SGD</code> （与线性回归完全相同）
训练	训练循环（对应 d2l 的 <code>train_ch3</code> ）

1. 加载 Fashion-MNIST 数据集

Fashion-MNIST 是服装分类数据集，包含 10 个类别：T 恤、裤子、套头衫、连衣裙、外套、凉鞋、衬衫、运动鞋、包、踝靴。

- 每张图像为 **28×28 的灰度图**，展平后是 784 维向量（ $28 \times 28 = 784$ ）；
- 训练集 60000 张，测试集 10000 张；
- 输入维度 $d = 784$ ，输出类别数 $q = 10$ 。

下面是 `d2l.load_data_fashion_mnist` 的**自包含 PyTorch 版本**（无需安装 d2l 库），首次运行会自动下载数据集（约 30 MB）到 `./data` 目录。

```
In [85]: import torch
import torchvision
from torch.utils import data
from torchvision import transforms

def load_data_fashion_mnist(batch_size, resize=None):
    """下载并加载 Fashion-MNIST 数据集（自包含版 d2l.load_data_fashion_mnist）"""
    trans = [transforms.ToTensor()]
    if resize:
        trans.insert(0, transforms.Resize(resize))
    trans = transforms.Compose(trans)
    mnist_train = torchvision.datasets.FashionMNIST(
        root="./data", train=True, transform=trans, download=True)
    mnist_test = torchvision.datasets.FashionMNIST(
        root="./data", train=False, transform=trans, download=True)
    # num_workers=0: 避免 Windows 多进程 DataLoader 的兼容性问题
    return (data.DataLoader(mnist_train, batch_size, shuffle=True, num_workers=0),
            data.DataLoader(mnist_test, batch_size, shuffle=False, num_workers=0))

batch_size = 256
train_iter, test_iter = load_data_fashion_mnist(batch_size)

# 查看一个训练小批量的形状
for X, y in train_iter:
    print('X 形状:', X.shape)    # torch.Size([256, 1, 28, 28]): 批量×通道×高×宽
    print('y 形状:', y.shape)    # torch.Size([256]): 每个样本的类别标签
    break
```

X 形状: torch.Size([256, 1, 28, 28])

y 形状: torch.Size([256])

2. 初始化模型参数

softmax 回归的输出层是一个**全连接层**，因此只需在 `Sequential` 中添加一个带有 10 个输出的全连接层。`Sequential` 在这里并不是必要的，但它是实现深度模型的基础。

注意两点：

1. **PyTorch 不会隐式地调整输入的形状**，因此在 `Linear` 前加了 `nn.Flatten()` 层，把 $(n, 1, 28, 28)$ 的输入展平成 $(n, 784)$ ；
2. 仍然以**均值 0、标准差 0.01** 的正态分布随机初始化权重 ($\mathbf{w} \sim \mathcal{N}(0, 0.01^2)$)。

$$\mathbf{O} = \mathbf{XW} + \mathbf{b}, \quad \mathbf{X} \in \mathbb{R}^{n \times 784}, \quad \mathbf{W} \in \mathbb{R}^{784 \times 10}, \quad \mathbf{b} \in \mathbb{R}^{1 \times 10}$$

```
In [86]: from torch import nn

# PyTorch 不会隐式地调整输入的形状。因此，
# 我们在线性层前定义了展平层 (flatten)，来调整网络输入的形状
net = nn.Sequential(nn.Flatten(), nn.Linear(784, 10))

def init_weights(m):
    if type(m) == nn.Linear:
        nn.init.normal_(m.weight, std=0.01)
```

```
net.apply(init_weights);

# 查看参数形状:  $W \in R^{10 \times 784}$ ,  $b \in R^{10}$ 
print('权重 W 形状:', net[1].weight.shape)
print('偏置 b 形状:', net[1].bias.shape)
```

权重 W 形状: torch.Size([10, 784])
偏置 b 形状: torch.Size([10])

3. 重新审视 Softmax 的实现（数值稳定性）

从数学上讲，“先算 softmax、再送入交叉熵损失”是完全合理的；但从**计算角度看**，指数可能造成数值稳定性问题。

上溢（overflow）

softmax 函数为 $\hat{y}_j = \frac{\exp(o_j)}{\sum_k \exp(o_k)}$ 。如果某些 o_k 数值非常大， $\exp(o_k)$ 可能超过数据类型容许的最大数字（float32 上限约 3.4×10^{38} ，即 $\exp(89)$ 附近），使分母或分子变为 `inf`，最后得到 0、`inf` 或 `nan` 的 $\hat{y}_j$ ——此时交叉熵没有明确定义。

技巧一：减去最大值（不改变结果）

在继续 softmax 计算之前，先从所有 o_k 中减去 $\max(o_k)$ 。每个 o_k 按同一常数移动不会改变 softmax 的返回值：

$$\begin{aligned}\hat{y}_j &= \frac{\exp(o_j - \max(o_k)) \exp(\max(o_k))}{\sum_k \exp(o_k - \max(o_k)) \exp(\max(o_k))} \\ &= \frac{\exp(o_j - \max(o_k))}{\sum_k \exp(o_k - \max(o_k))}\end{aligned}$$

减法后最大的指数项恰好是 $\exp(0) = 1$ ，上溢被消除。

下溢（underflow）

但减法后某些 $o_j - \max(o_k)$ 是较大的负数， $\exp(\cdot)$ 会得到接近零的值（下溢）；这些值四舍五入为零后， $\log(\hat{y}_j) = -\infty$ 。反向传播几步后，我们可能会发现自己面对一屏幕可怕的 `nan` 结果。

技巧二：LogSumExp —— 把 softmax 和交叉熵合在一起

我们最终要计算的是 $\log(\hat{y}_j)$ 。将两个步骤合并后， $\log(\exp(\cdot))$ 相互抵消，可以直接使用 o_j 本身：

$$\begin{aligned}\log(\hat{y}_j) &= \log\left(\frac{\exp(o_j - \max(o_k))}{\sum_k \exp(o_k - \max(o_k))}\right) \\ &= \underbrace{o_j - \max(o_k)}_{\text{不再取指数, 无上溢}} - \log\left(\sum_k \exp(o_k - \max(o_k))\right)\end{aligned}$$

因此，不要把 softmax 概率传入损失函数，而是直接传入未规范化的预测 (logits)，让 `nn.CrossEntropyLoss` 在内部“同时计算 softmax 及其对数”——这就是 [LogSumExp 技巧](#)。

```
In [87]: import torch
from torch import nn

# 演示 1: 直接计算 softmax 的上溢问题
o = torch.tensor([1000.0, 1001.0, 1002.0])
print('直接 exp(o)      :', torch.exp(o))           # 上溢 → [inf, inf, inf]
softmax_naive = torch.exp(o) / torch.exp(o).sum()
print('直接 softmax    :', softmax_naive)          # inf/inf → nan

# 演示 2: 减去最大值 — 不改变结果，且数值稳定
o_shift = o - o.max()
softmax_stable = torch.exp(o_shift) / torch.exp(o_shift).sum()
print('减 max 后 softmax:', softmax_stable)        # tensor([0.0900, 0.2447, 0.6652])

# 演示 3: CrossEntropyLoss 直接接收未规范化的预测 (Logits)
y = torch.tensor([0, 2])
Y_hat_logits = torch.tensor([[1000.0, 1001.0, 1002.0], # 故意用巨大的 logits
                             [ 5.0,   1.0,   3.0]])
loss = nn.CrossEntropyLoss(reduction='none')
print('CrossEntropyLoss :', loss(Y_hat_logits, y))  # 正常值，无 inf/nan
```

直接 exp(o) : tensor([inf, inf, inf])
直接 softmax : tensor([nan, nan, nan])
减 max 后 softmax : tensor([0.0900, 0.2447, 0.6652])
CrossEntropyLoss : tensor([2.4076, 2.1429])

4. 优化算法

使用学习率为 0.1 的小批量随机梯度下降作为优化算法。这与线性回归例子中使用的完全相同——同一个 `torch.optim.SGD` 就能训练分类模型，这体现了优化器的普适性。

```
In [88]: # 学习率为 0.1 的小批量随机梯度下降（与线性回归中使用的优化器完全相同）
trainer = torch.optim.SGD(net.parameters(), lr=0.1)
```

5. 训练模型

调用训练函数来训练模型（对应 d2l 的 `d2l.train_ch3`）。这里给出自包含的精简实现，逻辑与 d2l 完全一致：

- `Accumulator`：在多个变量上累加求和的小工具类；

- `accuracy`：统计预测正确的样本数（对多输出概率取 `argmax` 再比较）；
- `evaluate_accuracy`：在给定数据集上计算模型精度（`net.eval()` + `torch.no_grad()`）；
- `train_epoch_ch3`：一个 epoch 的训练——前向传播 → `CrossEntropyLoss` → 梯度清零 → 反向传播 → SGD 更新；
- `train_ch3`：逐 epoch 训练并记录损失/精度，最后绘制训练曲线（替代 d2l 的 `Animator`）。

```
In [89]: import matplotlib.pyplot as plt
# 中文字体设置: matplotlib 默认字体不含中文, 需指定 Windows 中文字体, 否则显示为方框
plt.rcParams['font.sans-serif'] = ['Microsoft YaHei', 'SimHei']
plt.rcParams['axes.unicode_minus'] = False # 正常显示负号

class Accumulator:
    """在 n 个变量上累加（对应 d2l.Accumulator）"""
    def __init__(self, n):
        self.data = [0.0] * n

    def add(self, *args):
        self.data = [a + float(b) for a, b in zip(self.data, args)]

    def reset(self):
        self.data = [0.0] * len(self.data)

    def __getitem__(self, idx):
        return self.data[idx]

def accuracy(y_hat, y):
    """统计预测正确的样本数"""
    if len(y_hat.shape) > 1 and y_hat.shape[1] > 1:
        y_hat = y_hat.argmax(axis=1)
    cmp = y_hat.type(y.dtype) == y
    return float(cmp.type(y.dtype).sum())

def evaluate_accuracy(net, data_iter):
    """计算模型在给定数据集上的精度"""
    net.eval() # 评估模式
    metric = Accumulator(2) # [正确数, 总数]
    with torch.no_grad():
        for X, y in data_iter:
            metric.add(accuracy(net(X), y), y.numel())
    return metric[0] / metric[1]

def train_epoch_ch3(net, train_iter, loss, updater):
    """训练一个迭代周期（epoch），返回（平均损失，训练精度）"""
    net.train() # 训练模式
    metric = Accumulator(3) # [损失总和, 正确数, 样本数]
    for X, y in train_iter:
        y_hat = net(X)
        l = loss(y_hat, y) # CrossEntropyLoss(reduction='none') → 每样本损
```

```

        updater.zero_grad()
        l.mean().backward()
        updater.step()
        metric.add(float(l.sum()), accuracy(y_hat, y), y.numel())
    return metric[0] / metric[2], metric[1] / metric[2]

def train_ch3(net, train_iter, test_iter, loss, num_epochs, trainer):
    """精简版 d2l.train_ch3: 训练并绘制损失与精度曲线（替代 Animator）"""
    history = {'train_loss': [], 'train_acc': [], 'test_acc': []}
    for epoch in range(num_epochs):
        train_loss, train_acc = train_epoch_ch3(net, train_iter, loss, trainer)
        test_acc = evaluate_accuracy(net, test_iter)
        history['train_loss'].append(train_loss)
        history['train_acc'].append(train_acc)
        history['test_acc'].append(test_acc)
        print(f'epoch {epoch + 1:2d}/{num_epochs} | '
              f'损失 {train_loss:.4f} | 训练精度 {train_acc:.4f} | 测试精度 {test_a

# 绘制训练曲线：损失（对数轴）与精度共用一个 epoch 轴
epochs = range(1, num_epochs + 1)
fig, ax1 = plt.subplots(figsize=(7, 4))
ax1.plot(epochs, history['train_loss'], 'o-', color='tab:red', label='训练损失')
ax1.set_xlabel('epoch')
ax1.set_ylabel('损失')
ax1.set_yscale('log')
ax2 = ax1.twinx()
ax2.plot(epochs, history['train_acc'], 's-', color='tab:blue', label='训练精度')
ax2.plot(epochs, history['test_acc'], '^-', color='tab:green', label='测试精度')
ax2.set_ylabel('精度')
lines = ax1.get_lines() + ax2.get_lines()
ax1.legend(lines, [line.get_label() for line in lines], loc='center right')
ax1.set_title('softmax 回归（简洁实现）的训练曲线')
plt.tight_layout()
plt.show()

```

```

In [90]: num_epochs = 10
         train_ch3(net, train_iter, test_iter, loss, num_epochs, trainer)

```

```

epoch 1/10 | 损失 0.7855 | 训练精度 0.7503 | 测试精度 0.7858
epoch 2/10 | 损失 0.5693 | 训练精度 0.8143 | 测试精度 0.8113
epoch 3/10 | 损失 0.5238 | 训练精度 0.8267 | 测试精度 0.8185
epoch 4/10 | 损失 0.5012 | 训练精度 0.8323 | 测试精度 0.8118
epoch 5/10 | 损失 0.4852 | 训练精度 0.8367 | 测试精度 0.8296
epoch 6/10 | 损失 0.4746 | 训练精度 0.8392 | 测试精度 0.8220
epoch 7/10 | 损失 0.4651 | 训练精度 0.8430 | 测试精度 0.8285
epoch 8/10 | 损失 0.4573 | 训练精度 0.8447 | 测试精度 0.8295
epoch 9/10 | 损失 0.4511 | 训练精度 0.8470 | 测试精度 0.8294
epoch 10/10 | 损失 0.4471 | 训练精度 0.8476 | 测试精度 0.8333

```

softmax 回归 (简洁实现) 的训练曲线

